UNIT-I

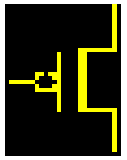
Faculty :Dr. Jisha P

SYLLABUS

Introduction: VLSI design flow, importance of HDLs, Verilog HDL and Design Methodologies, modules, instances, components of simulation, example, basic concepts. Modules and ports: Modules, ports, Rules.

EVIDENCE OF MOORE'S LAW

ACCORDING TO GORDEN MOORE -

YEAR	1996	MID 1997	1999	MID 2001
 (in millions)	5	10	20	40

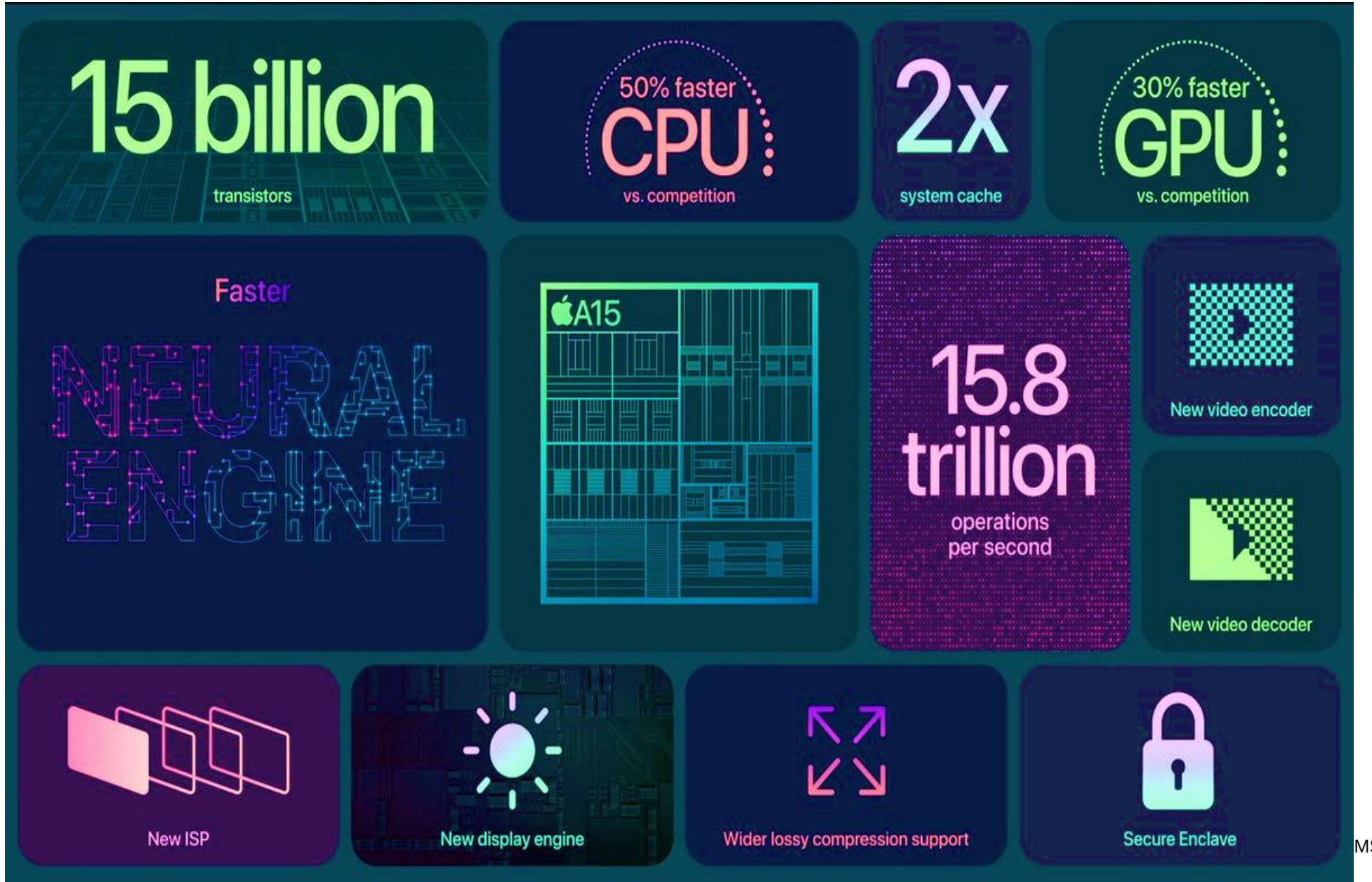
Pentium-I Came In 1996 **Transistor Count – 5 Million**

Pentium-IV Came In 2001 **Transistor Count – 43 Million**

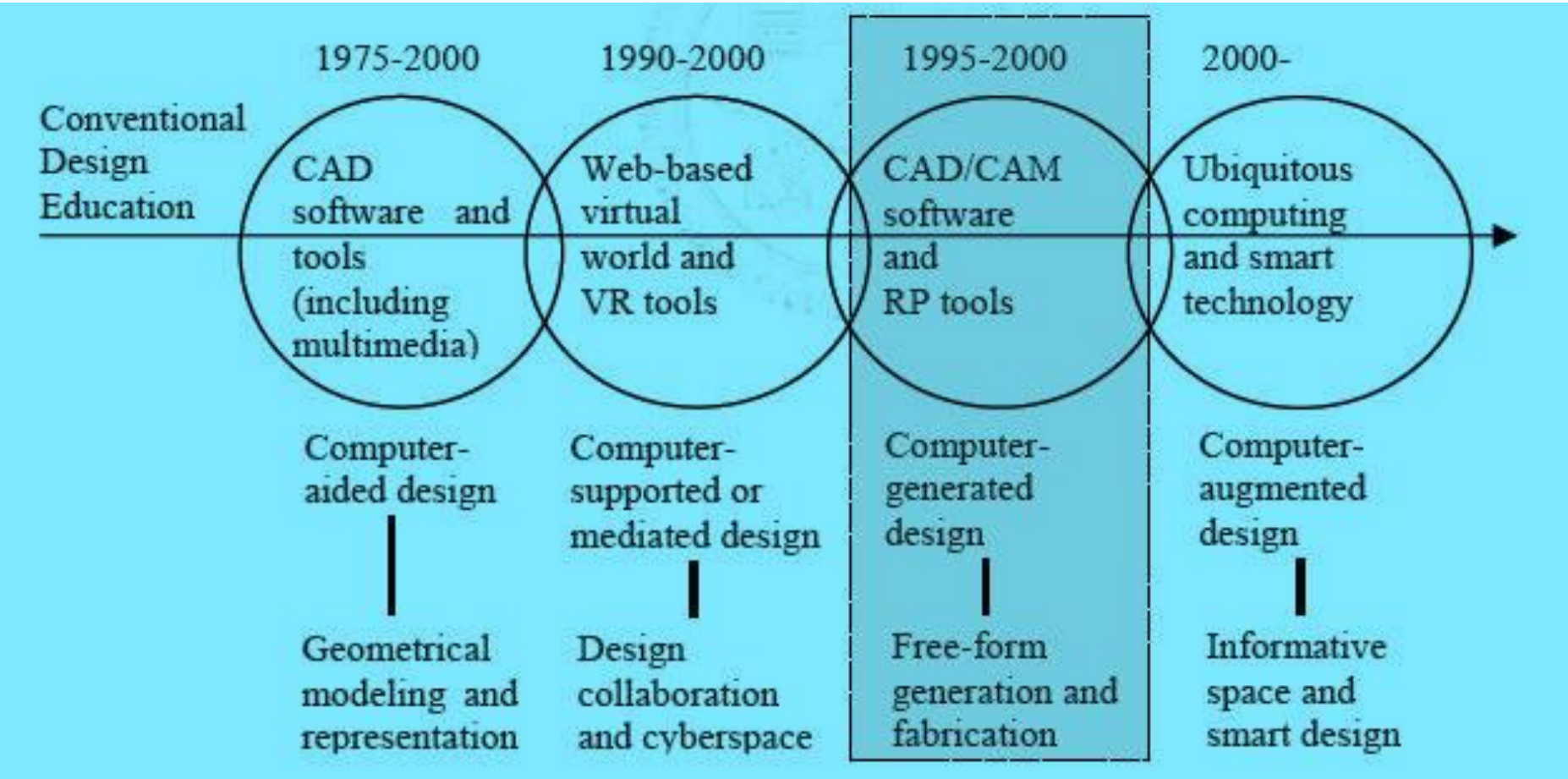
SOME MORE EXAMPLES !

	Count	Year Introduced
Quad-core+GPU Core i7 Ivy Bridge	1,400,000,000	2012
Quad-core+GPU Core i7 Haswell	1,400,000,000	2014
Quad-core+GPU GT2 Core i7 Skylake K	1,750,000,000	2015
Six-core Core i7 Ivy Bridge E	1,860,000,000	2013
Dual-core+GPU Iris Core i7 Broadwell-U	1,900,000,000	2015
Six-core Core i7 Sandy Bridge-E	2,270,000,000	2011
8-core Core i7 Haswell-E	2,600,000,000	2014
10-core Core i7 Broadwell-E	3,200,000,000	2016

Apple's A15 Bionic Processor- iPhone 13 Models



EVOLUTION OF COMPUTER-AIDED DIGITAL DESIGN



EMERGENCE OF HDLS

- HDLs allowed the designers to model the concurrency of processes found in hardware elements.
- Hardware description languages such as Verilog HDL and VHDL became popular.
- Verilog HDL originated in 1983 at Gateway Design Automation. Later, VHDL was developed under contract from DARPA. Both Verilog and VHDL simulators to simulate large digital circuits quickly gained acceptance from designers.
- Digital circuits could be described at a register transfer level (RTL) by use of an HDL.
- Thus, the designer had to specify how the data flows between registers and how the design processes the data.
- The details of gates and their interconnections to implement the circuit were automatically extracted by logic synthesis tools from the RTL description.

DESIGN METHODOLOGY

Top-down design

- Realizing the desired complex behavior of a H/W by partitioning it into an interconnect of simpler sub-behavior
- Create lower levels of abstraction from upper levels.

Bottom-up design

- Aims at realizing the desired complex behavior by suitable selection and interconnection of parts/components from an available set of parts/components with known behavior.
- Creates abstractions from low-level behavior.

Good design needs both top-down and bottom-up efforts !

INTRODUCTION: VLSI DESIGN FLOW

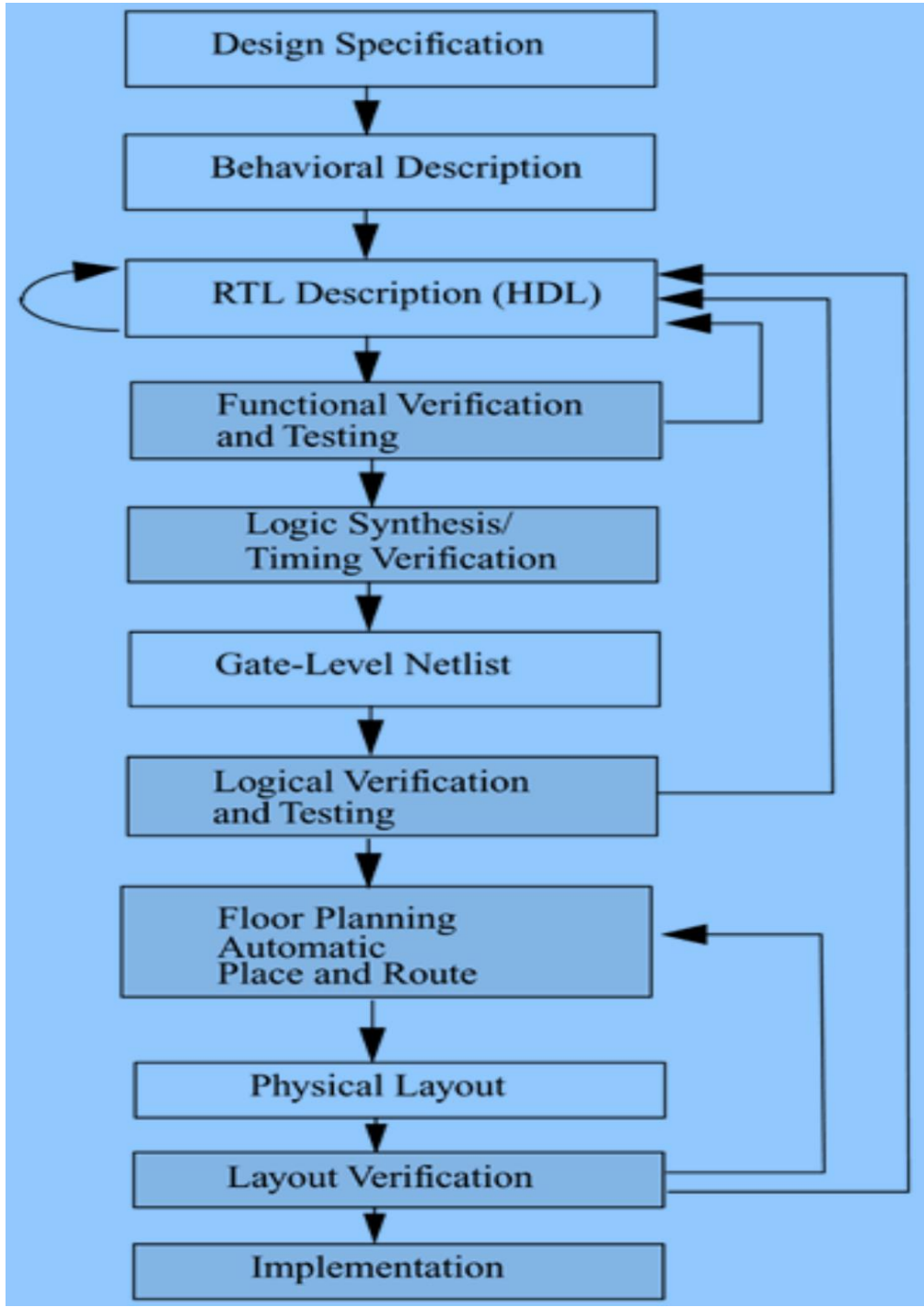
VLSI stands for Very Large Scale Integration where by Integration Circuits (ICs) are formed by inter VLSI stands for Very Large Scale Integration where by Integration Circuits (ICs) are formed by interconnecting a large number of components including resistors, transistors, capacitors and other components on a single chip. VLSI design is a process of designing these advanced ICs which are miniaturized and involves one or more intrinsic loops to generate the final product owning to certain specifications.

The VLSI design process includes several steps: software requirements, architectural design, behavioural or functional design, logical design, circuit design, physical design, exemplary design, fabrication and packaging, and testing and packaging. All of them are essential for the creation of the new IC starting from the idea and up to its turnkey functioning. Connecting a large number of components including resistors, transistors, capacitors and other components on a single chip. VLSI design is a process of designing these advanced ICs which are miniaturized and involves one or more intrinsic loops to generate the final product owning to certain specifications.

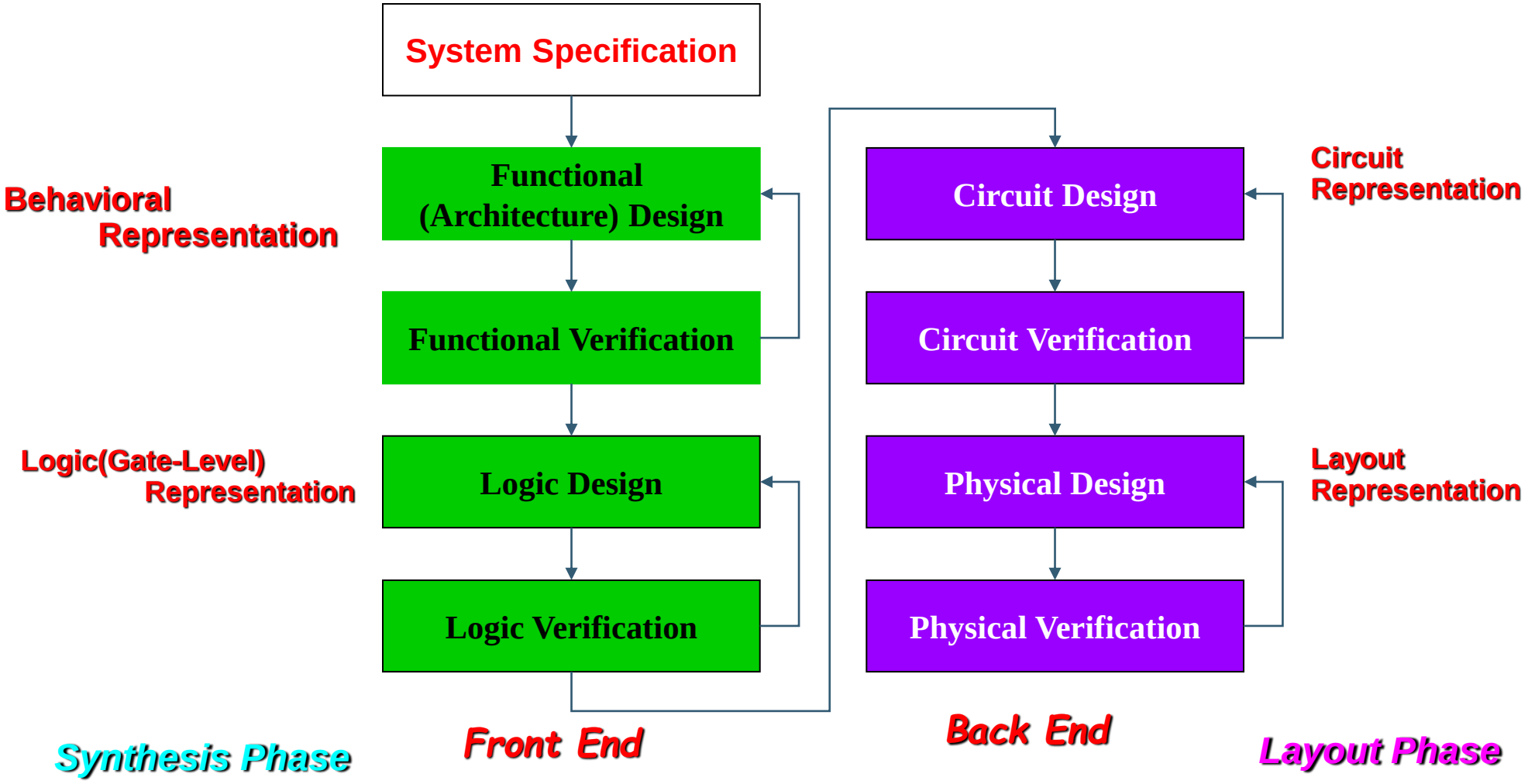
VLSI DESIGN FLOW

The VLSI design process includes several steps: software requirements, architectural design, behavioural or functional design, logical design, circuit design, physical design, exemplary design, fabrication and packaging, and testing and packaging.

VLSI Design is an iterative cycle. Designing a VLSI Chip includes a few problems such as functional design, logic design, circuit design, and physical design. The design is verified for accuracy by the process of simulation. If any design errors are found at any stage of verification, at least one of the previous design steps must be repeated to correct the error during the process of designing.



SIMPLIFIED VLSI DESIGN FLOW

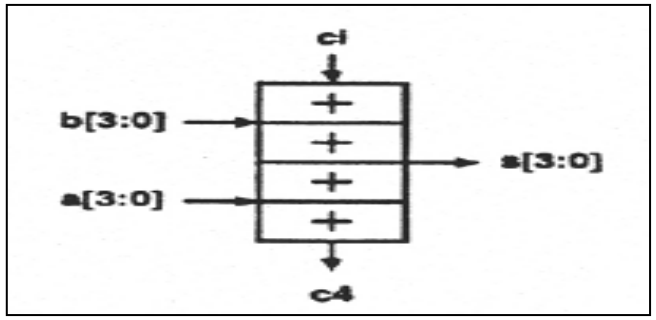


SIMPLIFIED VLSI DESIGN FLOW

- Specifications describe abstractly the functionality, interface, and overall architecture of the digital circuit to be designed.
- A behavioural description is then created to analyze the design in terms of functionality, performance, compliance to standards, and other high-level issues. **Behavioural descriptions are often written with HDLs.**
- The behavioural description is manually converted to an RTL description in an HDL. The designer has to describe the data flow that will implement the desired digital circuit.
- Logic synthesis tools convert the RTL description to a **gate-level netlist**.
- A gate-level netlist is a description of the circuit in terms of gates and connections between them. Logic synthesis tools ensure that the gate-level netlist meets timing, area, and power specifications.
- The gate-level netlist is input to an Automatic Place and Route tool, which creates a layout.
- The layout is verified and then fabricated on a chip.

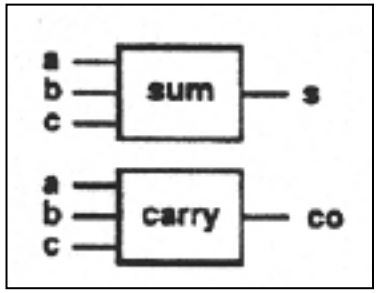
Four Levels of Design Representation

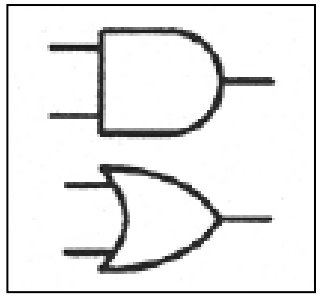
Behavioral Representation



Functional Blocks, FSM

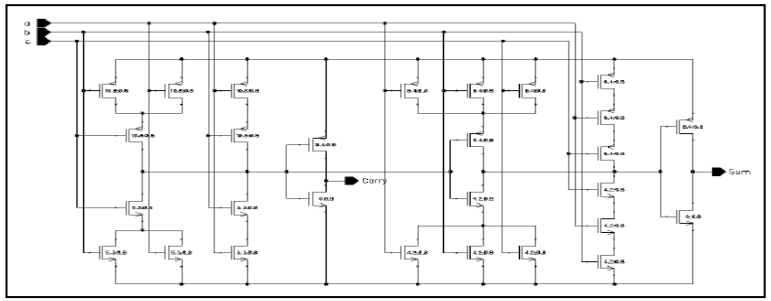
Logic (Gate-Level) Representation





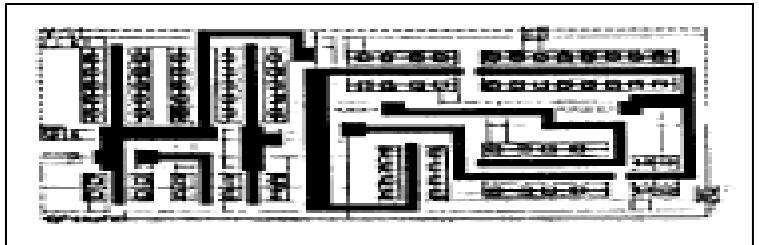
Logic Blocks, Gates

Circuit (Transistor-Level) Representation



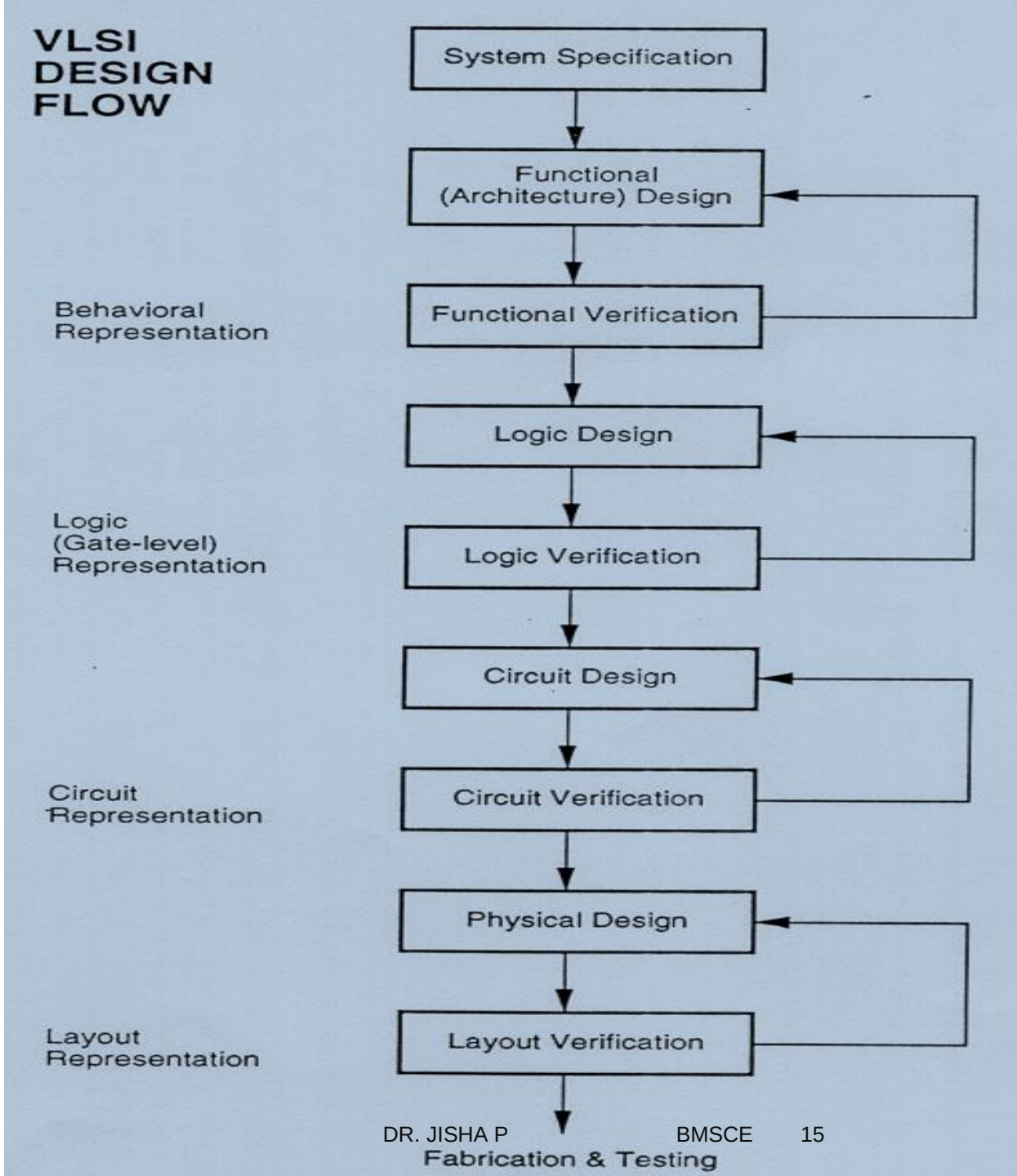
Transistor Schematics

Layout Representation



Physical Devices

VLSI DESIGN FLOW



IMPORTANCE OF HDLS

- Designs can be described at a very abstract level by use of HDLs.
- Designers can write their RTL description without choosing a specific fabrication technology.
- Logic synthesis tools can automatically convert the design to any fabrication technology.
- If a new technology emerges, designers do not need to redesign their circuit.
- They simply input the RTL description to the logic synthesis tool and create a new gate-level netlist, using the new fabrication technology.
- The logic synthesis tool will optimize the circuit in area and timing for the new technology.

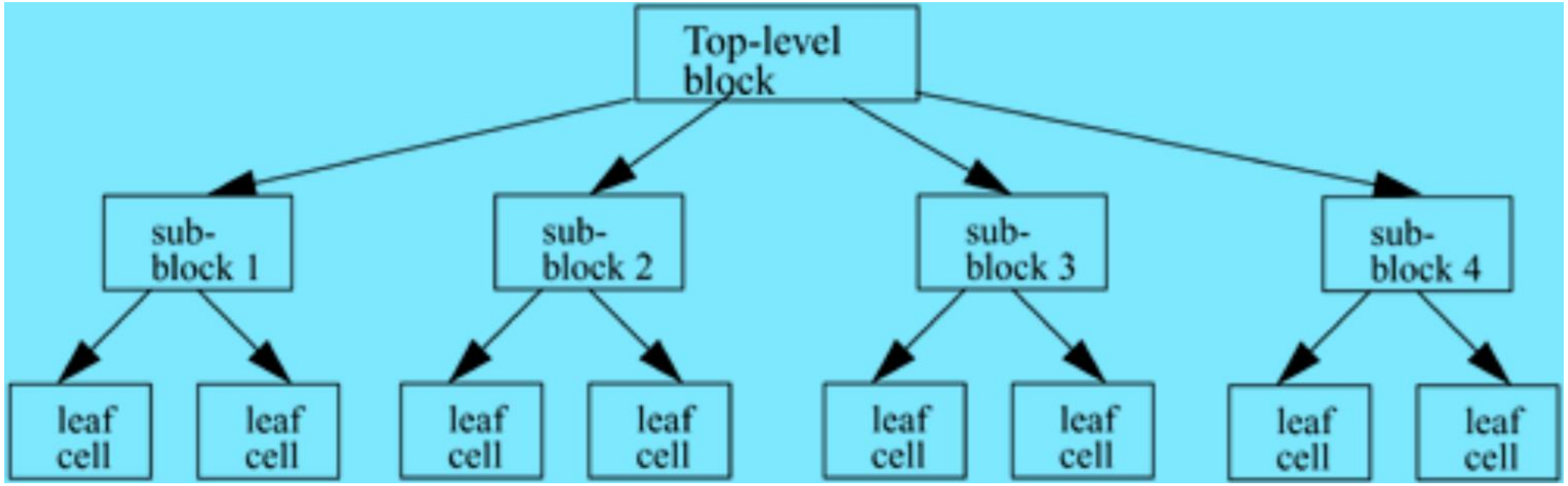
IMPORTANCE OF HDLS

- By describing designs in HDLs, functional verification of the design can be done early in the design cycle.
- Since designers work at the RTL level, they can optimize and modify the RTL description until it meets the desired functionality. Most design bugs are eliminated at this point.
- This cuts down design cycle time significantly because the probability of hitting a functional bug at a later time in the gate-level netlist or physical layout is minimized.
- Designing with HDLs is analogous to computer programming.
- A textual description with comments is an easier way to develop and debug circuits.
- This also provides a concise representation of the design, compared to gate-level schematics.
- Gate-level schematics are almost incomprehensible for very complex designs.

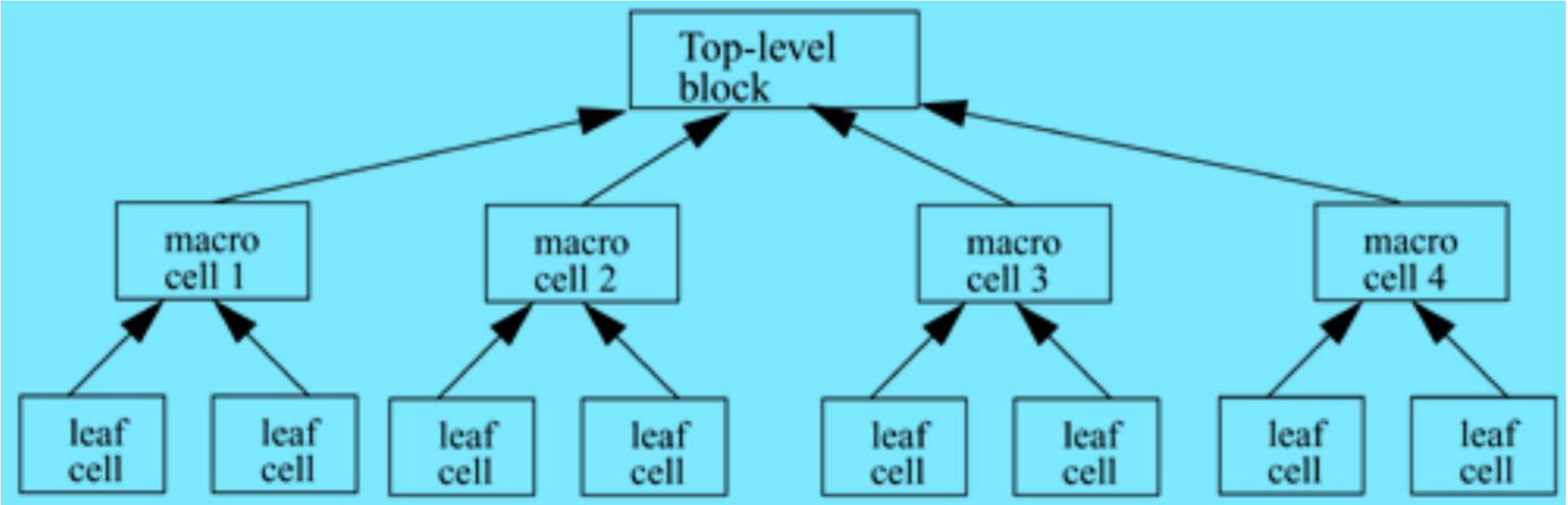
VERILOG HDL AND DESIGN METHODOLOGIES

- There are two basic types of digital design methodologies:
  - **a top-down design methodology**
  - **a bottom-up design methodology.**
- In a top-down design methodology, we define the top-level block and identify the sub-blocks necessary to build the top-level block.
- We further subdivide the sub-blocks until we come to leaf cells, which are the cells that cannot further be divided.
- In a bottom-up design methodology, we first identify the building blocks that are available to us.
- We build bigger cells, using these building blocks.
- These cells are then used for higher-level blocks until we build the top-level block in the design.

TOP-DOWN DESIGN METHODOLOGY



BOTTOM-UP DESIGN METHODOLOGY

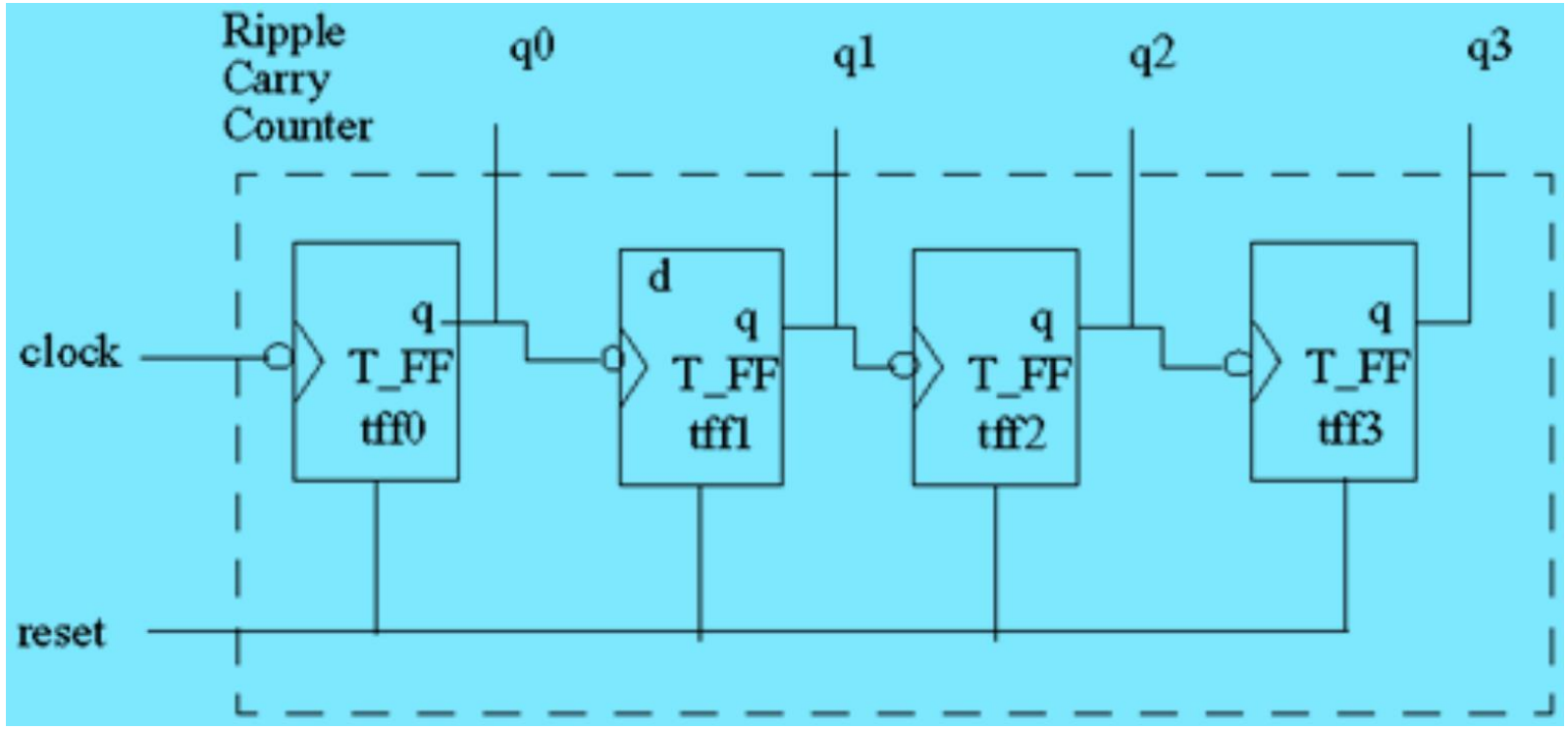


VERILOG HDL AND DESIGN METHODOLOGIES

- Typically, a combination of top-down and bottom-up flows is used.
- Design architects define the specifications of the top-level block.
- Logic designers decide how the design should be structured by breaking up the functionality into blocks and sub-blocks.
- At the same time, circuit designers are designing optimized circuits for leaf-level cells.
- They build higher-level cells by using these leaf cells.
- The flow meets at an intermediate point where the switch-level circuit designers have created a library of leaf cells by using switches, and the logic level designers have designed from top-down until all modules are defined in terms of leaf cells.

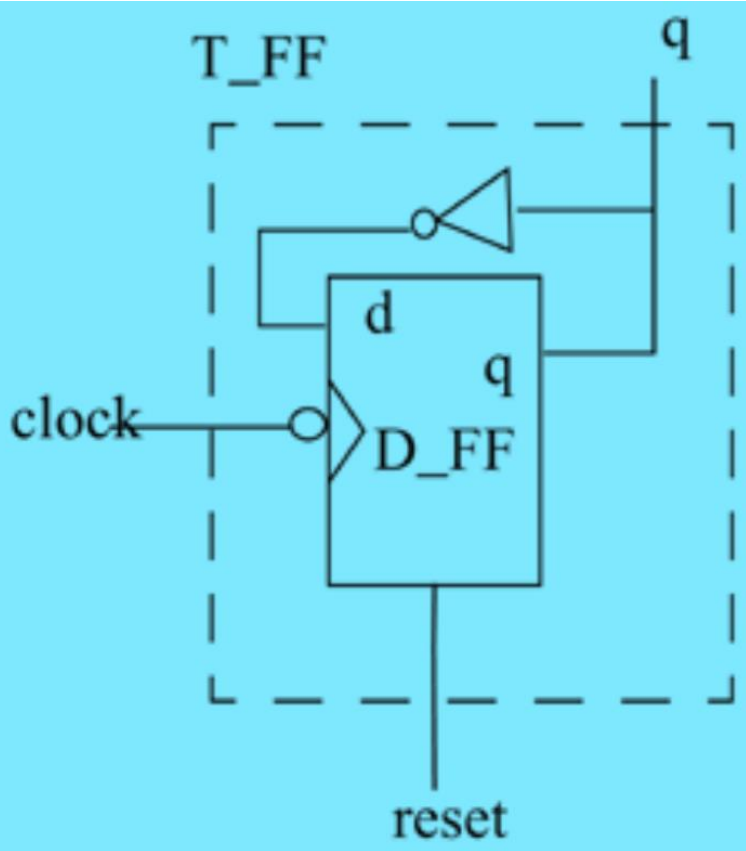
EG:- 4-BIT RIPPLE CARRY COUNTER

- The ripple carry counter shown in Figure is made up of negative edge-triggered toggle flipflops (T_FF).
- Each of the T_FFs can be made up from negative edge-triggered D-flipflops (D_FF) and inverters (assuming q_bar output is not available on the D_FF),

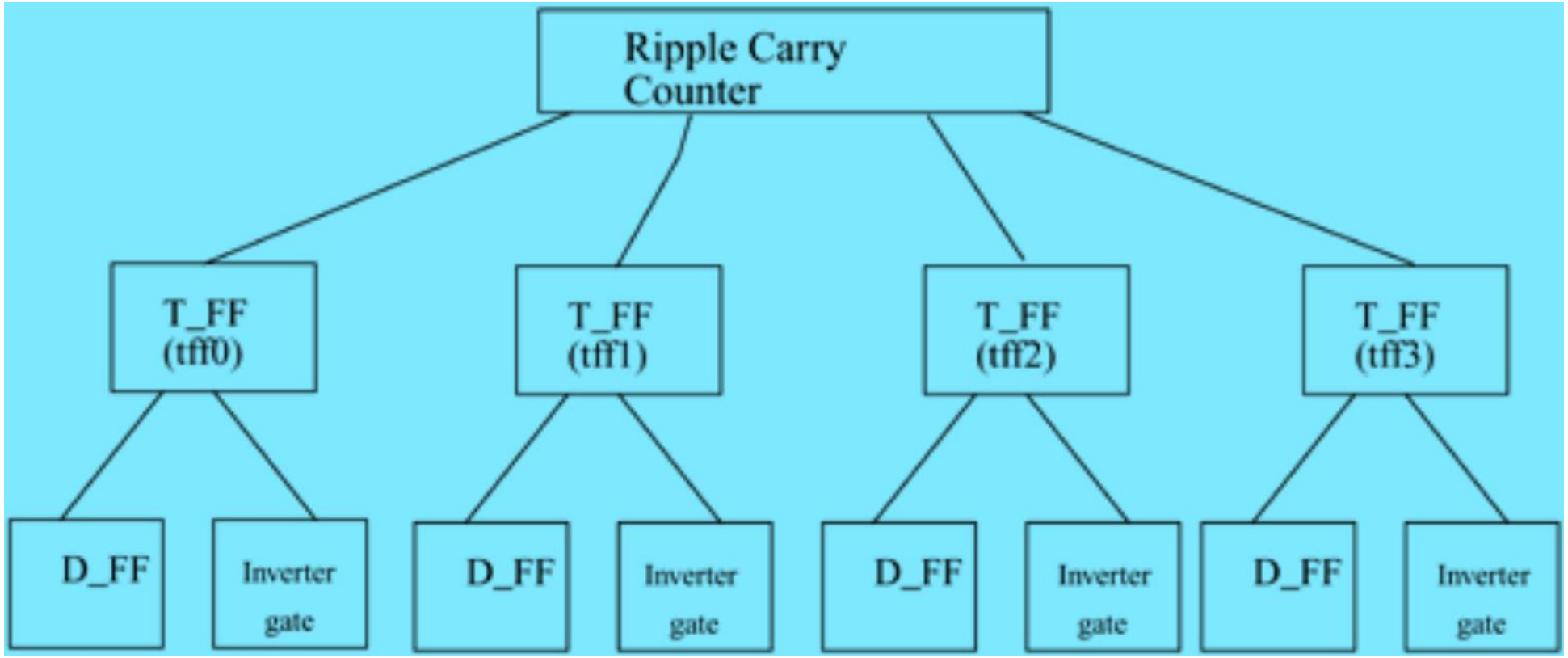


EG:- **4-BIT RIPPLE CARRY COUNTER**

reset	q_n	q_{n+1}
1	1	0
1	0	0
0	0	1
0	1	0
0	0	0



EG:- **4-BIT RIPPLE CARRY COUNTER**



Thus, the ripple carry counter is built in a hierarchical fashion by using building blocks. The diagram for the design hierarchy is shown in Figure.

EG:- **4-BIT RIPPLE CARRY COUNTER**

- In a top-down design methodology, we first have to specify the functionality of the ripple carry counter, which is the top-level block.
- Then, we implement the counter with T_FFs.
- We build the T_FFs from the D_FF and an additional inverter gate.
- Thus, we break bigger blocks into smaller building sub-blocks until we decide that we cannot break up the blocks any further.
- A bottom-up methodology flows in the opposite direction.
- We combine small building blocks and build bigger blocks; e.g., we could build D_FF from and and or gates, or we could build a custom D_FF from transistors.
- Thus, the bottom-up flow meets the top-down flow at the level of the D_FF.

MODULES

- A module is the basic building block in Verilog.
- **A module can be an element or a collection of lower-level design blocks.**
- Typically, elements are grouped into modules to provide common functionality that is used at many places in the design.
- A module provides the necessary functionality to the higher-level block through its port interface (inputs and outputs), but hides the internal implementation.
- This allows the designer to modify module internals without affecting the rest of the design.
- In Verilog, a module is declared by the keyword **module**. A corresponding keyword **endmodule** must appear at the end of the module definition.
- **Each module must have a module_name, which is the identifier for the module**, and a **module_terminal_list**, which describes the input and output terminals of the module.

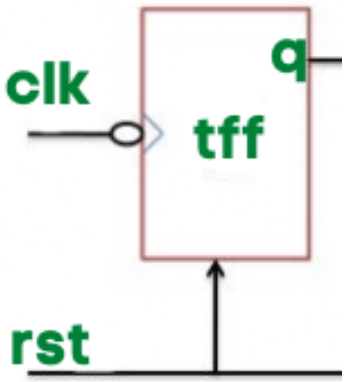
MODULES – EG:

```
module <module_name>
(<module_terminal_list>);

...

<module internals>

...

...

endmodule
```

In ripple carry counter, T_FF, D_FF are examples of modules.

The T-flipflop could be defined as a module as follows:

```
module T_FF (q, clock, reset);

.

.

<functionality of T-flipflop>

.

.

endmodule
```



MODULES

- Verilog is both a behavioural and a structural language.
- Internals of each module can be defined at four levels of abstraction, depending on the needs of the design.
- The module behaves identically with the external environment irrespective of the level of abstraction at which the module is described.
- The internals of the module are hidden from the environment.
- Thus, the level of abstraction to describe a module can be changed without any change in the environment.

MODULES -LEVEL OF ABSTRACTION

- **Behavioural or algorithmic level**

This is the **highest level of abstraction** provided by Verilog HDL. A module can be implemented in terms of the desired design algorithm without concern for the hardware implementation details. Designing at this level is very similar to C programming.

- **Dataflow level**

At this level, the module is designed by specifying the data flow. The designer is aware of how data flows between hardware registers and how the data is processed in the design.

- **Gate level**

The module is implemented in terms of logic gates and interconnections between these gates. Design at this level is similar to describing a design in terms of a gate-level logic diagram.

- **Switch level**

This is the lowest level of abstraction provided by Verilog. A module can be implemented in terms of switches, storage nodes, and the interconnections between them. Design at this level requires knowledge of switch-level implementation details.

MODULES -LEVEL OF ABSTRACTION

- Verilog allows each of the modules to be written at a different level of abstraction.
- As the design matures, most modules are replaced with gate-level implementations.
- Normally, the higher the level of abstraction, the more flexible and technology independent the design.
- As one goes lower toward switch-level design, the design becomes technology-dependent and inflexible.
- ie. A small modification can cause a significant number of changes in the design.

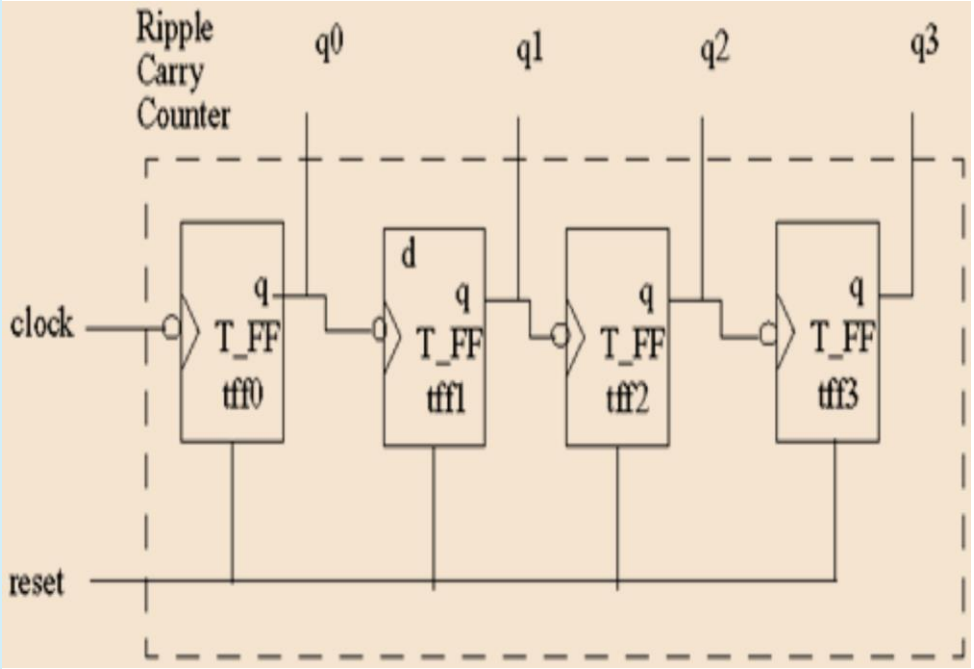
INSTANCES

- A module provides a template from which you can create actual objects.
- When a module is invoked, Verilog creates a unique object from the template.
- Each object has its own name, variables, parameters, and I/O interface.
- **The process of creating objects from a module template is called instantiation, and the objects are called instances.**
- In Ripple counter example the top-level block creates four instances from the T-flipflop (T_FF) template.
- Each T_FF instantiates a D_FF and an inverter gate.
- Each instance must be given a unique name.

EXAMPLE: MODULE INSTANTIATION

**NOTE THAT // IS USED TO DENOTE SINGLE-LINE COMMENTS.
//DEFINE THE TOP-LEVEL MODULE CALLED RIPPLE CARRY COUNTER.
//IT INSTANTIATES 4 T-FLIPFLOPS. INTERCONNECTIONS ARE // SHOWN IN 4-BIT RIPPLE CARRY COUNTER.

```
module ripple_carry_counter(q, clk, reset);
output [3:0] q; // I/O signals and vector declarations
input clk, reset;
//Four instances of the module T_FF are created.
//Each has a unique name. Each instance is passed a set of signals.
//Notice, that each instance is a copy of the module T_FF.
T_FF tff0(q[0],clk, reset);
T_FF tff1(q[1],q[0], reset);
T_FF tff2(q[2],q[1], reset);
T_FF tff3(q[3],q[2], reset);
endmodule
```



EXAMPLE: MODULE INSTANTIATION – CONTD..

// Define the module T_FF. It instantiates a d-flipflop.
// We assumed that module d-flipflop is defined elsewhere in the design.

```
module T_FF(q, clk, reset);
output q;
input clk, reset;
wire d;
D_FF dff0(q, d, clk, reset); // Instantiate D_FF. Call it dff0.
not n1(d, q);        // not gate is a Verilog primitive.
endmodule
```

INSTANCES

- In Verilog, it is illegal to nest modules.
- One module definition cannot contain another module definition within the module and endmodule statements.
- Instead, a module definition can incorporate copies of other modules by instantiating them.
- Module definitions simply specify how the module will work, its internals, and its interface.
- **Modules must be instantiated for use in the design**.

EXAMPLE: ILLEGAL MODULE NESTING

```
// Define the top-level module called ripple carry counter.
// It is illegal to define the module T_FF inside this module.
module ripple_carry_counter(q, clk, reset);
output [3:0] q;
input clk, reset;
module T_FF(q, clock, reset); // ILLEGAL MODULE NESTING
...
<module T_FF internals>
...
endmodule // END OF ILLEGAL MODULE NESTING
endmodule
```

COMPONENTS OF A SIMULATION

Once a design block is completed, it must be tested.

The functionality of the design block can be tested by applying stimulus and checking results.

We call such a block the stimulus block.

It is good practice to keep the stimulus and design blocks separate.

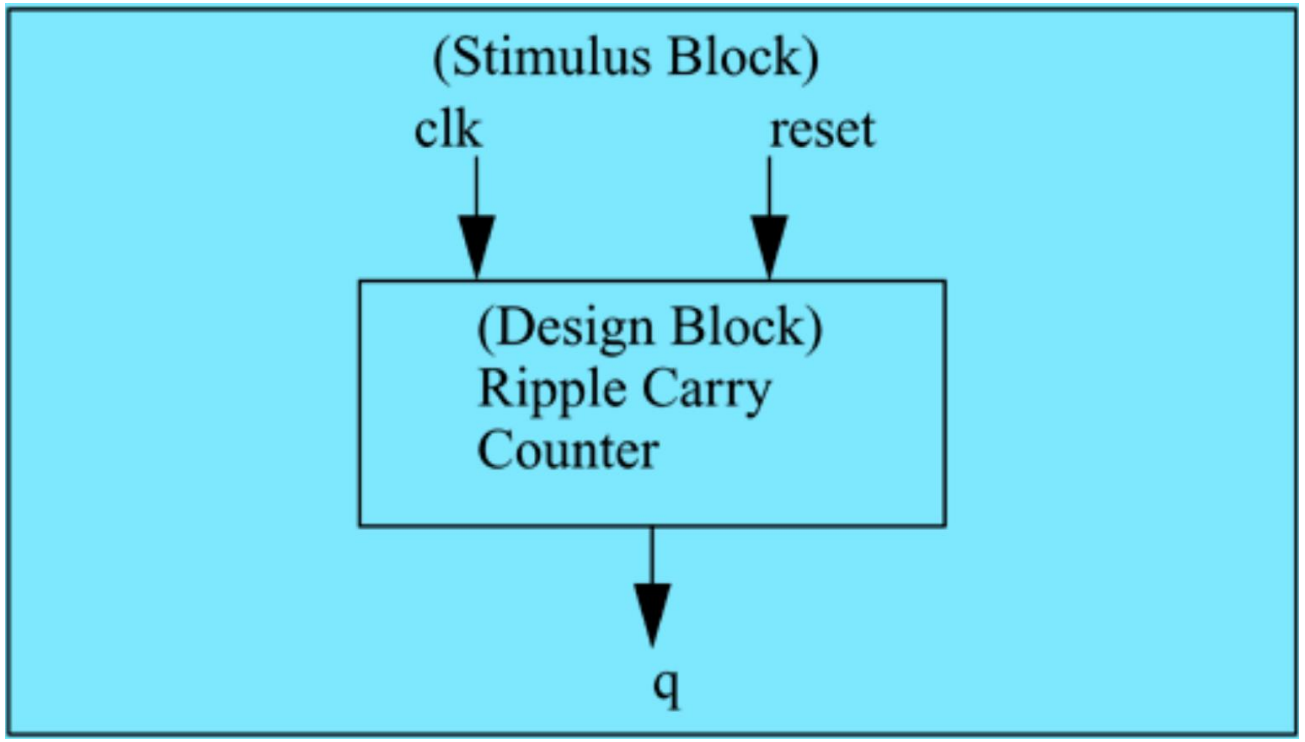
The stimulus block can be written in Verilog. A separate language is not required to describe stimulus.

The stimulus block is also commonly called a test bench.

Different test benches can be used to thoroughly test the design block.

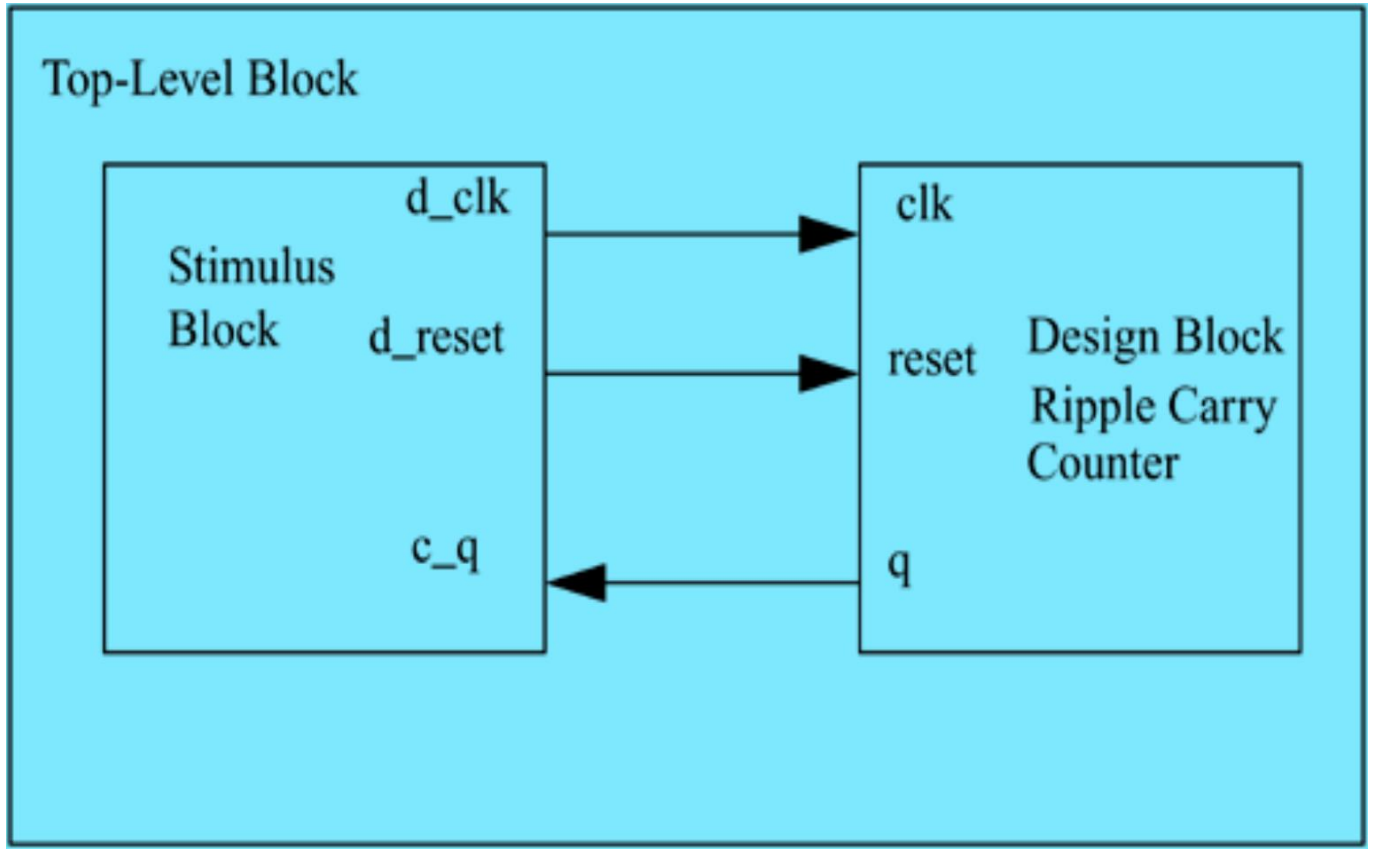
COMPONENTS OF A SIMULATION

- Two styles of stimulus application are possible.
- In the first style, the stimulus block instantiates the design block and directly drives the signals in the design block.

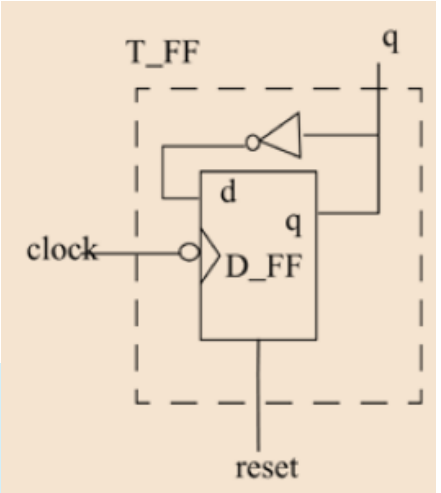


COMPONENTS OF A SIMULATION

- The second style of applying stimulus is to instantiate both the stimulus and design blocks in a top-level dummy module.
- The stimulus block interacts with the design block only through the interface
- Either stimulus style can be used effectively.



EXAMPLE - A TOP-DOWN DESIGN METHODOLOGY



Example 1: Ripple Carry Counter Top Block

```
module ripple_carry_counter(q, clk, reset);
output [3:0] q;
input clk, reset;
//4 instances of the module t_ff are created.
T_FF tff0(q[0],clk, reset);
T_FF tff1(q[1],q[0], reset);
T_FF tff2(q[2],q[1], reset);
T_FF tff3(q[3],q[2], reset);
endmodule
```

Example 2- Flipflop T_FF

```
module T_FF(q, clk, reset);
output q;
input clk, reset;
wire d;
D_FF dff0(q, d, clk, reset);
not n1(d, q); // not is a Verilog-provided primitive. case sensitive
endmodule
```

EXAMPLE - A TOP-DOWN DESIGN METHODOLOGY

Since T_FF instantiates D_FF, we must now define (example 2) the internals of module D_FF. We assume synchronous reset for the D_FFF.

Example 2-5 Flipflop D_F

```
// module D_FF with synchronous reset
module D_FF(q, d, clk, reset);
output q;
input d, clk, reset;
reg q;
always @(posedge reset or negedge clk)
if (reset)
q <= 1'b0;
else
q <= d;
endmodule
```

All modules have been defined down to the lowest-level leaf cells in the design methodology. The design block is now complete

EXAMPLE - **STIMULUS BLOCK**

- In this case, we must control the signals clk and reset so that the regular function of the ripple carry counter and the asynchronous reset mechanism are both tested.
- We use the waveforms shown in Figure below to test the design.
- Waveforms for clk, reset, and 4-bit output q are shown.
- The cycle time for clk is 10 units;
- The reset signal stays up from time 0 to 15 and then goes up again from time 195 to 205.
- Output q counts from 0 to 15.

EXAMPLE 1: RIPPLE CARRY COUNTER TOP BLOCK

```
module ripple_carry_counter(q, clk, reset);
output [3:0] q;
input clk, reset;
//4 instances of the module T_FF are created.
T_FF tff0(q[0],clk, reset);
T_FF tff1(q[1],q[0], reset);
T_FF tff2(q[2],q[1], reset);
T_FF tff3(q[3],q[2], reset);
endmodule
```

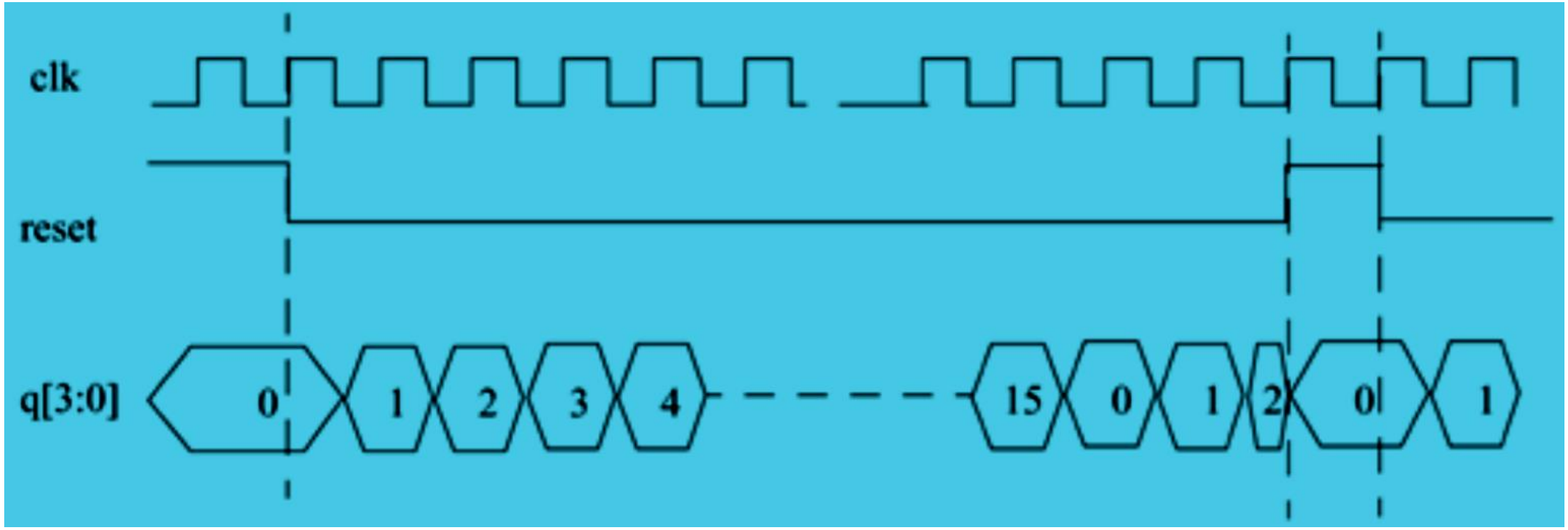
EXAMPLE: STIMULUS BLOCK (TEST BENCH)

```
module stimulus;
reg clk;
reg reset;
wire[3:0] q;
// instantiate the design block
ripple_carry_counter r1(q, clk, reset);
// Control the clk signal that drives the
//design block. Cycle time = 10
initial
clk = 1'b0; //set clk to 0
always
#5 clk = ~clk; //toggle clk every 5 time units
```

```
// Control the reset signal that drives the design block
// reset is asserted from 0 to 20 and from 200 to 220.
initial
begin
reset = 1'b1;
#15 reset = 1'b0;
#180 reset = 1'b1;
#10 reset = 1'b0;
#20 $finish; //terminate the simulation
end
// Monitor the outputs
initial
$monitor($time, " Output q = %d", q);
endmodule
```

EXAMPLE - STIMULUS BLOCK

STIMULUS AND OUTPUT WAVEFORMS



EXERCISE- 1

1. An interconnect switch (IS) contains the following components, a shared memory (MEM), a system controller (SC) and a data crossbar (Xbar).

A. Define the modules MEM, SC, and Xbar, using the module/endmodule keywords. You do not need to define the internals. Assume that the modules have no terminal lists.

B. Define the module IS, using the module/endmodule keywords. Instantiate the modules MEM, SC, Xbar and call the instances mem1,sc1, and xbar1, respectively. You do not need to define the internals. Assume that the module IS has no terminals.

C. Define a stimulus block (Top), using the module/endmodule keywords. Instantiate the design block IS and call the instance is1. This is the final step in building the simulation environment.

ANSWER

A. Defining the modules MEM, SC, and Xbar.

```
module MEM;
    // Internal logic can be defined here.
endmodule

module SC;
    // Internal logic can be defined here.
endmodule

module Xbar;
    // Internal logic can be defined here.
endmodule
```

ANSWER

B. Defining the module IS and instantiating mem1, sc1, and xbar1

```
module IS;
    // Instantiating the modules
    MEM mem1();
    SC sc1();
    Xbar xbar1();
endmodule
```

ANSWER

C. Defining the top-level stimulus block Top and instantiating IS

```
module Top;
    // Instantiating the IS module
    IS is1();
endmodule
```

EXERCISE- 2

2: A 4-bit ripple carry adder (Ripple_Add) contains four 1-bit full adders (FA).

A. Define the module FA. Do not define the internals or the terminal list.

B. Define the module Ripple_Add. Do not define the internals or the terminal list. Instantiate four full adders of the type FA in the module Ripple_Add and call them fa0, fa1, fa2, and fa3.

ANSWER

A. Defining the module FA (1-bit Full Adder)

```
module FA;
    // Internal logic and terminal list can be defined here.
endmodule
```

B. Defining the module Ripple_Add and instantiating four full adders (fa0, fa1, fa2, and fa3)

```
module Ripple_Add;
    // Instantiating four 1-bit full adders
    FA fa0();
    FA fa1();
    FA fa2();
    FA fa3();
endmodule
```

BASIC CONCEPTS - LEXICAL CONVENTIONS

- The basic lexical conventions used by Verilog HDL are similar to those in the C programming language.
- Verilog contains a stream of tokens.
- Tokens can be comments, delimiters, numbers, strings, identifiers, and keywords.
- Verilog HDL is a case-sensitive language. All keywords are in lowercase.

LEXICAL CONVENTIONS

- Whitespace
  - Blank spaces (\b) , tabs (\t) and newlines (\n) comprise the whitespace. Whitespace is ignored by Verilog except when it separates tokens.
  - Whitespace is not ignored in strings.

- Comments
  - Comments can be inserted in the code for readability and documentation.
  - There are two ways to write comments. A one-line comment starts with "//". Verilog skips from that point to the end of line.
  - A multiple-line comment starts with "/*" and ends with "*/".
  - Multiple-line comments cannot be nested.
  - However, one-line comments can be embedded in multiple-line comments.

LEXICAL CONVENTIONS

- Operators

  - Operators are of three types: unary, binary, and ternary.
  - Unary operators precede the operand.
  - Binary operators appear between two operands.
  - Ternary operators have two separate operators that separate three operands.

- `a = ~ b; // ~ is a unary operator. b is the operand`
- `a = b && c; // && is a binary operator. b and c are operands`
- `a = b ? c : d; // ?: is a ternary operator. b, c and d are operands`

LEXICAL CONVENTIONS

```
4'b1111 // This is a 4-bit binary number
12'habc // This is a 12-bit hexadecimal number
16'd255 // This is a 16-bit decimal number.
```

- Number Specification
  - There are two types of number specification in Verilog: **sized and unsized.**
  - **Sized numbers** Sized numbers are represented as:

<size> '<base format> <number>

  - <size> is written only in decimal and specifies the number of bits in the number.
  - Legal base formats are decimal ('d or 'D), hexadecimal ('h or 'H), binary ('b or 'B) and octal (‘o or 'O).
  - The number is specified as consecutive digits from 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f.
  - Only a subset of these digits is legal for a particular base.
  - Uppercase letters are legal for number specification.

LEXICAL CONVENTIONS

- Unsized numbers
  - Numbers that are specified without a <base format> specification are **decimal numbers** by default.
  - Numbers that are written without a <size> specification have a default number of bits that is simulator- and machine-specific (must be at least **32 bit**).

```
23456 // This is a 32-bit decimal number by
default
'hc3 // This is a 32-bit hexadecimal number
'o21 // This is a 32-bit octal number
```

LEXICAL CONVENTIONS

```
12'h13x // This is a 12-bit hex number; 4 least significant bits unknown

6'hx // This is a 6-bit hex number

32'bz // This is a 32-bit high impedance number
```

- **X or Z values**
  - Verilog has two symbols for unknown and high impedance values. These values are very important for modelling real circuits.
  - An unknown value is denoted by an x. A high impedance value is denoted by z.
  - An x or z sets four bits for a number in the hexadecimal base, three bits for a number in the octal base, and one bit for a number in the binary base.
  - If the most significant bit of a number is 0, x, or z, the number is automatically extended to fill the most significant bits, respectively, with 0, x, or z.

LEXICAL CONVENTIONS

- Negative numbers
- Negative numbers can be specified by putting a minus sign before the size for a constant number.
- Size constants are always positive. It is illegal to have a minus sign between <base format> and <number>.
- An optional signed specifier can be added for signed arithmetic.

```
• -6'd3 // 8-bit negative number stored as 2's complement of 3
• -6'sd3 // Used for performing signed integer math
• 4'd-2 // Illegal specification !
```

LEXICAL CONVENTIONS

- Underscore characters and question marks
- An underscore character "_" is allowed anywhere in a number except the first character.
- **Underscore characters are allowed only to improve readability of numbers and are ignored by Verilog**.
- A question mark **"?" is the Verilog HDL alternative for z in the context of numbers.**
- The ? is used to enhance readability in the casex and casez statements.

```
• 12'b1111_0000_1010 // Use of underline characters for readability
• 4'b10?? // Equivalent of a 4'b10zz
```

LEXICAL CONVENTIONS

- Strings
  - A string is a sequence of characters that are enclosed by double quotes.
  - The restriction on a string is that it must be contained on a single line, that is, without a carriage return.
  - It cannot be on multiple lines.
  - Strings are treated as a sequence of one-byte ASCII values.

```
• "Hello Verilog World" // is a string
• "a / b" // is a string
```

LEXICAL CONVENTIONS

- `reg value; // reg is a keyword; value is an identifier`
- `input clk; // input is a keyword, clk is an identifier`

Identifiers and Keywords :

- **Keywords are special identifiers reserved to define the language constructs.**
- Keywords are in lowercase.
- **Identifiers are names given to objects so that they can be referenced in the design**.
- Identifiers are made up of alphanumeric characters, the underscore (_), or the dollar sign ($).
- Identifiers are case sensitive. **Identifiers start with an alphabetic character or an underscore. They cannot start with a digit or a $ sign.**
- Escaped identifiers begin with the backslash (\ ) character and end with whitespace (space, tab, or newline).
- All characters between backslash and whitespace are processed literally.
- Any printable ASCII character can be included in escaped identifiers.
- Neither the backslash nor the terminating whitespace is considered to be a part of the identifier.

DATA TYPES

- **Value Set**
- Verilog supports four values and eight strengths to model the functionality of real hardware.
- In addition to logic values, strength levels are often used to resolve conflicts between drivers of different strengths in digital circuits.

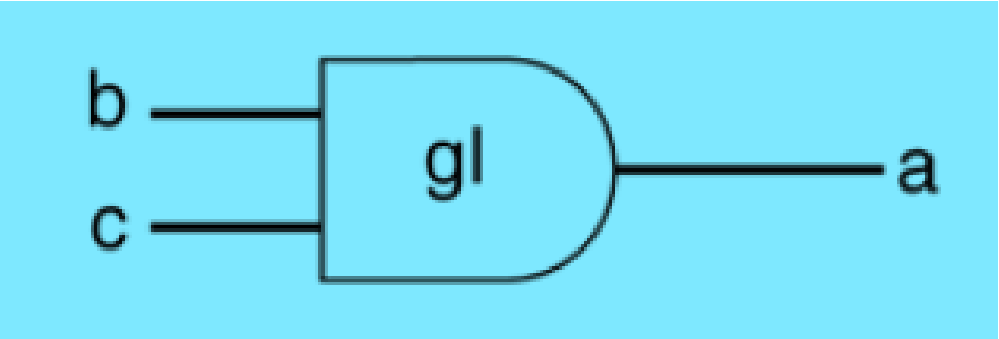
Strength Levels

Strength Level	Type	Degree
supply	Driving	strongest
strong	Driving	
pull	riving	
large	Storage	
weak	Driving	
medium	Storage	
small	Storage	
highz	High Impedance	weakest

Table 3-1. Value Levels

Value Level	Condition in Hardware Circuits
0	Logic zero, false condition
1	Logic one, true condition
x	Unknown logic value
z	High impedance, floating state

DATA TYPES



- Nets
- Nets represent connections between hardware elements.
- Just as in real circuits, **nets have values continuously driven on them by the outputs of devices that they are connected to.**
- Nets are declared primarily with the keyword wire.
- Nets are one-bit values by default unless they are declared explicitly as vectors.
- **The terms wire and net are often used interchangeably**.
- The default value of a net is z (except the tri reg net, which defaults to x).
- Nets get the output value of their drivers. If a net has no driver, it gets the value z.
- Note that net is not a keyword **but represents a class of data types such as wire, wand, wor, tri, triand, trior, trireg**, etc.

```
• wire a;
• // Declare net a for the
  above circuit
• wire b,c;
• // Declare two wires b,c for
  the above circuit
• wire d = 1'b0;
• // Net d is fixed to logic
  value 0 at declaration.
```

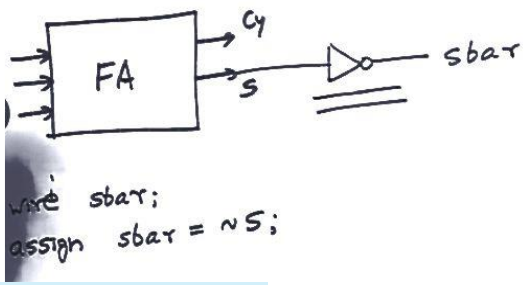
DATA TYPES

Do not confuse the term registers in Verilog with hardware registers built from edge-triggered flipflops in real circuits !

- Registers
- Registers represent data storage elements.
- **Registers retain value until another value is placed onto them.**
- In Verilog, the term register merely means a variable that can hold a value.
- **Unlike a net, a register does not need a driver.**
- Verilog registers do not need a clock as hardware registers do.
- Values of registers can be changed anytime in a simulation by assigning a new value to the register.
- **Register data types are commonly declared by the keyword reg.**
- The default value for a reg data type is x.
- Registers can also be declared as signed variables. Such registers can be used for signed arithmetic.

```
wire sbar;
assign sbar = ~S;
```

A VARIABLE CAN BE:



A. Net - wire, wor, wand, tri, supply0, supply1, etc.

- Must be continuously driven,
- Cannot be used to store a value,
- Models connections between continuous assignments and instantiations,
- 1-bit values by default, unless declared as vectors explicitly.
- Default value of a *net* is “Z” - high impedance state.

B. Register

- Retains the last value assigned to it,
- Usually used to represent storage elements (sometimes in combinational circuits),
- May or may not map to a HW register during synthesis.
- Default value of a *reg* data type is “X”.

WIRE vs REG

There are two types of variables in Verilog:

- Wires (all outputs of assign statements must be wires)
- Regs (all outputs of always blocks must be regs)

Both variables can be used as inputs anywhere

- Can use regs or wires as inputs (RHS) to assign statements

```
assign bus = LatchOutput + ImmediateValue
// bus must be a wire, but LatchOutput can be a reg
```

- Can use regs or wires as inputs (RHS) in always blocks

```
always @ (in or clk)
  if (clk) out = in     // in can be a wire, out must be a reg
```

DATA TYPES

Example 3-1 Example of Register

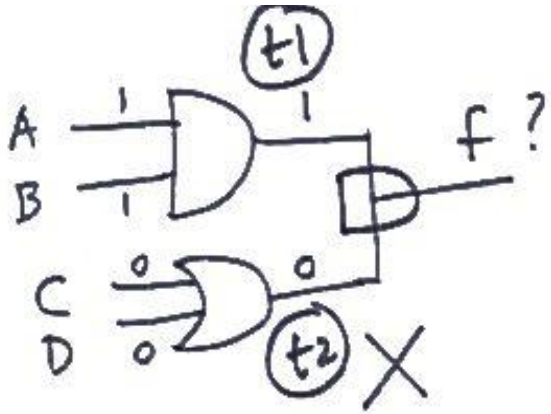
```
reg reset; // declare a variable reset that can hold its value

initial // this construct will be discussed later

begin

reset = 1'b1; //initialize reset to 1 to reset the digital circuit.

#100 reset = 1'b0; // after 100 time units reset is deasserted.

end
```

Example 3-2 Signed Register Declaration

```
reg signed [63:0] m; // 64 bit signed value

integer i; // 32 bit signed value
```

NET EXAMPLE:1

```
module use_wire(a, b, c, d, f);
    input a, b, c, d;
    output f;
    wire f; // net f declared as wire
    assign f = a & b;
    assign f = c | d;
endmodule
```



$f = t1 \& t2$

Here, function realized will be
f = (A & B) & (C | D)

NET EXAMPLE:2

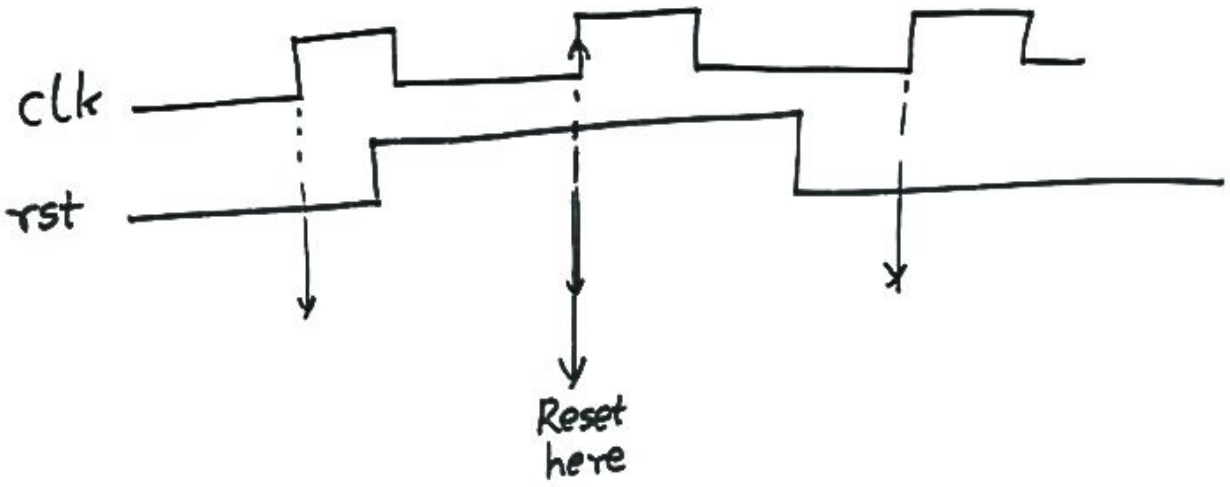
```
// A 2-level combinational circuit
module using_supply_wire(a, b, c, f);
    input a, b, c;
    output f;
    wire t1, t2;
    supply0 gnd;
    supply1 vcc;
    nand G1 (t1, vcc, a, b);
    xor G2 (t2, c, gnd);
    and G3 (f, t1, t2);
endmodule
```

REG EXAMPLE:

- Declaration explicitly specifies the size (default is 1-bit):
  - `reg x, y; // 1-bit register variables`
  - `reg [7:0] bus; // An 8-bit bus`
- Treated as an unsigned number in arithmetic expressions.
- **MUST** be used when modeling actual ***sequential*** HW, e.g. counters, shift registers, etc.
- Two types of assignments possible:
  - `A = B + C;`
  - `A <= B + C;`

REG EXAMPLE - 32-BIT COUNTER WITH SYNCHRONOUS RESET

```
module simple_counter(clk, rst, count);
    input clk, rst;
    output [31:0] count;
    reg [31:0] count;

    always @(posedge clk)
    begin
        if(rst)
            count = 32'b0;
        else
            count = count + 1;
    end
endmodule
```



DATA TYPES

Vectors:

- Nets or reg data types can be declared as vectors (multiple bit widths). If bit width is not specified, the default is scalar (1-bit).
- Vectors can be declared at **[high# : low#] or [low# : high#],** but the left number in the squared brackets is always the most significant bit of the vector.

```
wire a; // scalar net variable, default

wire [7:0] bus; // 8-bit bus

wire [31:0] busA,busB,busC; // 3 buses of 32-bit width.

reg clock; // scalar register, default

reg [0:40] virtual_addr; // Vector register, virtual address 41 bits wide
```

DATA TYPES

Vector Part Select :

- It is possible to address bits or parts of vectors.
  - busA[7] // bit # 7 of vector busA
  - bus[2:0] // Three least significant bits of vector bus
- Variable Vector Part Select:
- This allows part selects to be put in for loops to select various parts of the vector.
- There are two special part-select operators:

  [<starting_bit>+:width] - part-select increments from starting bit

  [<starting_bit>-:width] - part-select decrements from starting bit

  - The starting bit of the part select can be varied, but the width has to be constant.

DATA TYPES

Vector Part Select :

```
//Using a variable part select, one can choose parts
byte = data1[31-:8]; //starting bit = 31, width =8 => data[31:24]
byte = data1[24+:8]; //starting bit = 24, width =8 => data[31:24]
byte = data2[31-:8]; //starting bit = 31, width =8 => data[24:31]
byte = data2[24+:8]; //starting bit = 24, width =8 => data[24:31]
```

- The starting bit can also be a variable. The width has to be constant.
- Therefore, one can use the variable part select in a loop to select all bytes of the vector.

```
for (j=0; j<=31; j=j+1)
byte = data1[(j*8)+:8]; //Sequence is [7:0], [15:8]... [255:248]
//Can initialize a part of the vector
data1[(byteNum*8)+:8] = 8'b0; //If byteNum = 1, clear 8 bits [15:8]
```

DATA TYPES

INTEGER , REAL, AND TIME REGISTER DATA TYPES

- **Integer**
- An integer is a general purpose register data type used for manipulating quantities.
- Integers are declared by the keyword integer.
- Although it is possible to use reg as a general-purpose variable, it is more convenient to declare an integer variable for purposes such as counting.
- The default width for an integer is the host-machine word size, which is implementation-specific but is at least 32 bits.
- Registers declared as data type reg store values as unsigned quantities, whereas integers store values as signed quantities.

```
integer counter; // general purpose variable used as a counter.

initial

counter = -1; // A negative one is stored in the counter
```

DATA TYPES
REAL

- Real number constants and real register data types are declared with the keyword real.
- They can be specified in decimal notation (e.g., 3.14) or in scientific notation (e.g., 3e6, which is 3 x 106).
- Real numbers cannot have a range declaration, and their default value is 0.
- When a real value is assigned to an integer, the real number is rounded off to the nearest integer.

```
real delta; // Define a real variable called delta
initial
begin
delta = 4e10; // delta is assigned in scientific notation
delta = 2.13; // delta is assigned a value 2.13
end
integer i; // Define an integer i
initial
i = delta; // i gets the value 2 (rounded value of 2.13)
```

DATA TYPES
TIME

- Verilog simulation is done with respect to simulation time.
- A special time register data type is used in Verilog to store simulation time.
- A time variable is declared with the keyword time.
- The width for time register data types is implementation-specific but is at least 64 bits.
- The system function $time is invoked to get the current simulation time.
- Simulation time is measured in terms of simulation seconds. The unit is denoted by s, the same as real time.

```
time save_sim_time; // Define a time variable save_sim_time

initial

save_sim_time = $time; // Save the current simulation time
```

Feature	Integer	Real	Time
Usage	Signed integer numbers	Floating-point numbers	Simulation time values
Declared as	integer	real	time
Bit Width	32-bit (signed)	Double-precision (64-bit)	Platform-dependent (64-bit)
Value Range	-2^31 to 2^31-1	IEEE 754 floating point format	Depends on simulator (usually 64-bit unsigned)
Typical Usage	Counters, loop indices	Mathematical calculations, precise measurements	Represent simulation time
Precision	Exact integer values	Approximate, floating-point precision	Exact values for time calculations
Operations Supported	Arithmetic (+, -, *, /), Bitwise operations	Arithmetic (+, -, *, /), Comparisons	Time-related calculations, comparison
Initialization Default	0	0.0	0
System Task Example	$display("%d", my_int);	$display("%f", my_real);	$display("%t", my_time);
Usage in Simulation	Useful in loops and counters	Calculations requiring precision, delay calculations	Simulation event tracking, time delays
Common System Tasks	$display, $monitor, $strobe	$realtime, $display, $monitor	$time, $display, $monitor

DATA TYPES - ARRAYS

- Arrays are allowed in Verilog for **reg, integer, time, real, real time and vector register** data types.
- Multi-dimensional arrays can also be declared with any number of dimensions.
- Arrays of nets can also be used to connect ports of generated instances.
- Each element of the array can be used in the same fashion as a scalar or vector net.
- Arrays are accessed by **<array_name>[<subscript>].**
- For multi-dimensional arrays, indexes need to be provided for each dimension.
- It is important not to confuse arrays with net or register vectors.
- A vector is a single element that is n-bits wide. On the other hand, arrays are multiple elements that are 1-bit or n-bits wide.

ARRAYS - DECLARATION

```
integer count[0:7]; // An array of 8 count variables

reg bool[31:0]; // Array of 32 one-bit boolean register variables

time chk_point[1:100]; // Array of 100 time checkpoint variables

reg [4:0] port_id[0:7]; // Array of 8 port_ids; each port_id is 5 bits wide

integer matrix[4:0][0:255]; // Two dimensional array of integers

reg [63:0] array_4d [15:0][7:0][7:0][255:0]; //Four dimensional array

wire [7:0] w_array2 [5:0]; // Declare an array of 8 bit vector wire

wire w_array1[7:0][5:0]; // Declare an array of single bit wires
```

ARRAYS – EXAMPLES OF ASSIGNMENTS TO ELEMENTS OF ARRAYS

```
count[5] = 0; // Reset 5th element of array of count variables
chk_point[100] = 0; // Reset 100th time check point value
port_id[3] = 0; // Reset 3rd element (a 5-bit value) of port_id array.
matrix[1][0] = 33559; // Set value of element indexed by [1][0] to
                          33559
array_4d[0][0][0][0][15:0] = 0; //Clear bits 15:0 of the register
                                //accessed by indices [0][0][0][0]
port_id = 0; // Illegal syntax!- Attempt to write the entire array
matrix [1] = 0; // Illegal syntax! - Attempt to write [1][0]..[1][255]
reg mem1bit[0:1023]; // Memory mem1bit with 1K 1-bit words
reg [7:0] membyte[0:1023]; // Memory membyte with 1K 8-bit words(bytes)
```

DATA TYPES - PARAMETERS

```
parameter port_id = 5; // Defines a constant port_id
parameter cache_line_width = 256; // Constant defines width of cache line
parameter signed [15:0] WIDTH; // Fixed sign and range for parameter
'WIDTH'
```

- Verilog allows constants to be defined in a module by the keyword parameter.
- Parameters cannot be used as variables.
- Parameter values for each module instance can be overridden individually at compile time.
- This allows the module instances to be customized.
- Module definitions may be written in terms of parameters.
- Hardcoded numbers should be avoided.
- Parameters values can be changed at module instantiation
- Verilog **HDL local parameters** (defined using keyword localparam -) are identical to parameters except that they cannot be directly modified with the defparam statement or by the ordered or named parameter value assignment.
- The localparam keyword is used to define parameters when their values should not be changed.

Parameters

- Constants with a given name,
- Size deduced from the constant value itself:

```
parameter HI = 5, LO = 0;
parameter up = 2'b00, down = 2'b01, steady = 2'b10;
```

```
// Parameterized design:
// an N-bit counter
module counter(clk, rst, count);
    parameter N = 31;
    input clk, rst;
    output [0:N] count;
    reg [0:N] count;

    always @(negedge clk)
    begin
        if (rst)
            count = 0;
        else
            count = count + 1;
    end
endmodule
```

Activate Windows
Go to Settings to activate

DATA TYPES - STRINGS

- Strings can be stored in reg.
- The width of the register variables must be large enough to hold the string.
- Each character in the string takes up 8 bits (1 byte).
- If the width of the register is greater than the size of the string, Verilog fills bits to the left of the string with zeros.
- If the register width is smaller than the string width, Verilog truncates the leftmost bits of the string.
- It is always safe to declare a string that is slightly wider than necessary.
- Special characters serve a special purpose in displaying strings, such as newline, tabs, and displaying argument values.
- **Special characters can be displayed in strings only when they are preceded by escape characters**

```
reg [8*18:1] string_value; // Declare a variable that is 18 bytes wide
initial
string_value = "Hello Verilog World"; // String can be stored in variable
```

DATA TYPES - STRINGS

Table 3-3. Special Characters

Escaped Characters	Character Displayed
\n	newline
\t	tab
%%	%
\\	\
\"	"
\ooo	Character written in 1?3 octal digits

Dataflow Statements & Behavioural Blocks

1. Dataflow (Assign) Statements:

- Dataflow modelling in Verilog describes the behaviour of the circuit using **continuous assignments**.
- These assignments are done with the **assign** keyword.
- It represents how data moves through the circuit and how outputs are generated based on inputs.
- The assign statement is used to drive values onto nets (e.g., wires).
- These assignments are combinational, meaning they occur continuously as inputs change.
- **Dataflow modelling is typically used for combinational logic**, and assign statements should not be used for sequential logic (where timing is involved).

```
Eg:- assign out = a & b;
// Output is always the result of ANDing 'a' and 'b'
```

Dataflow Statements & Behavioural Blocks

2. Behavioural Blocks (Always or Initial):

Behavioural modelling in Verilog uses procedural blocks like always and initial to describe the behaviour of the circuit in a more algorithmic or sequential way.

a) always Block:

The always block allows you to describe a circuit's behaviour that depends on some event, like changes in input signals or the clock.

Inside an always block, you can describe combinational or sequential logic.

Combinational Logic: **If the block depends on input signal changes, it's often used to describe combinational logic.**

Sequential Logic: **If the block is triggered by a clock edge, it's used to describe sequential logic (e.g., flip-flops or registers).**

Dataflow Statements & Behavioural Blocks

- **Syntax Example (Combinational):**

```
always @(a or b)

begin

    out = a & b;  // Perform AND

//operation whenever 'a' or 'b' changes

end
```

- **Syntax Example (Sequential):**

```
always @(posedge clk)
begin
    if (reset)
        q <= 0;     // Reset the output
    else
        q <= d;     // Store the value of 'd' into 'q' at
                    //every clock cycle
end
```

Dataflow Statements & Behavioural Blocks

b. initial Block:

The initial block is used to specify actions that should happen only once at the start of the simulation.

It's typically used for test benches, signal initialization, or for driving stimulus into the design.

Unlike always blocks, which repeat their actions as conditions change, initial blocks run their actions only once, when simulation begins.

Syntax Example:

```
initial begin
    a = 0;  // Initialize 'a' to 0 at the start of simulation
    #10 a = 1;  // After 10 time units, change 'a' to 1
end
```

Dataflow (assign): Continuously evaluates combinational expressions.

Behavioural (always or initial): Describes behaviour with sequential or combinational logic. always can be triggered by events like changes in signals or clock edges, while initial is used to set up initial conditions or test sequences.

SYSTEM TASKS AND COMPILER DIRECTIVES

System Tasks:

- Verilog provides standard system tasks for certain routine operations.
- All system tasks appear in the form **$<keyword>.**
- **Operations such as displaying on the screen, monitoring values of nets, stopping, and finishing are done by system tasks**.

SYSTEM TASKS- DISPLAYING INFORMATION

- **$display** is the main system task for displaying values of variables or strings or expressions. This is one of the most useful tasks in Verilog.
- Usage: $display(p1, p2, p3,....., pn);
- p1, p2, p3,..., pn can be quoted strings or variables or expressions.
- The format of $display is very similar to printf in C.
- A $display inserts a newline at the end of the string by default.
- A $display without any arguments produces a newline.

STRING FORMAT SPECIFICATIONS

Format	Display
%d or %D	Display variable in decimal
%b or %B	Display variable in binary
%s or %S	Display string
%h or %H	Display variable in hex
%c or %C	Display ASCII character
%m or %M	Display hierarchical name (no argument required)
%v or %V	Display strength
%o or %O	Display variable in octal
%t or %T	Display in current time format
%e or %E	Display real number in scientific format (e.g., 3e10)
%f or %F	Display real number in decimal format (e.g., 2.13)
%g or %G	Display real number in scientific or decimal, whichever is shorter

EXAMPLES OF THE $DISPLAY TASK.

IF VARIABLES CONTAIN X OR Z VALUES, THEY ARE PRINTED IN THE DISPLAYED STRING AS "X" OR "Z".

//Display the string in quotes

```
$display("Hello Verilog World");
```

-- Hello Verilog World

//Display value of current simulation time 230

```
$display($time);
```

-- 230

//Display value of 41-bit virtual address 1fe0000001c at time 200

```
reg [0:40] virtual_addr;
$display("At time %d virtual address is %h", $time, virtual_addr);
```

-- At time 200 virtual address is 1fe0000001c

//Display value of port_id 5 in binary

```
reg [4:0] port_id;
$display("ID of the port is %b", port_id);
```

-- ID of the port is 00101

SYSTEM TASKS- MONITORING INFORMATION

- Verilog provides a mechanism to monitor a signal when its value changes.
- This facility is provided by the **$monitor** task.
- Usage: **$monitor(p1,p2,p3,....,pn);**
- The parameters p1, p2, ... , pn can be variables, signal names, or quoted strings.
- A format similar to the $display task is used in the $monitor task.
- **$monitor continuously monitors the values of the variables or signals specified in the parameter list and displays all**
- parameters in the list whenever the value of any one variable or signal changes.
- Unlike $display, $monitor needs to be invoked only once.
- Only one monitoring list can be active at a time. If there is more than one $monitor statement in your simulation, the last $monitor statement will be the active statement. The earlier $monitor statements will be overridden.
- Two tasks are used to switch monitoring on and off.
- Usage: **$monitoron;** **and** **$monitoroff;**

EXAMPLE - MONITOR STATEMENT

```
//Monitor time and value of the signals clock and reset
//Clock toggles every 5 time units and reset goes down at 10 time units
initial
begin
$monitor($time," Value of signals clock = %b reset = %b", clock,reset);
end
```

Partial output of the monitor statement:

```
-- 0 Value of signals clock = 0 reset = 1
-- 5 Value of signals clock = 1 reset = 1
-- 10 Value of signals clock = 0 reset = 0
```

SYSTEM TASKS- STOPPING AND FINISHING IN A SIMULATION

- The task $stop is provided to stop during a simulation.
- **Usage: $stop;**
- The $stop task puts the simulation in an interactive mode.
- The designer can then debug the design from the interactive mode.
- The $stop task is used whenever the designer wants to suspend the simulation and examine the values of signals in the design.
- The $finish task terminates the simulation.
- **Usage: $finish;**

COMPILER DIRECTIVES

Compiler directives are provided in Verilog. All compiler directives are defined by using the `<keyword> construct.

The two most useful compiler directives:

1. **`define**

- The **`define** directive is used to define text macros in Verilog.
- The Verilog compiler substitutes the text of the macro wherever it encounters a
- `<macro_name>.
- This is similar to the **#define** construct in C.
- The defined constants or text macros are used in the Verilog code by preceding them with a ` (**back tick**).

COMPILER DIRECTIVES

Example : `define Directive

```
//define a text macro that defines default word size
//Used as 'WORD_SIZE in the code
'define WORD_SIZE 32
//define an alias. A $stop will be substituted wherever 'S appears
'define S $stop;
//define a frequently used text string
'define WORD_REG reg [31:0]
// you can then define a 32-bit register as 'WORD_REG reg32;
```

COMPILER DIRECTIVES

2. `include

- The **`include** directive allows you to include entire contents of a Verilog source file in another Verilog file during compilation.
- This works similarly to the **#include** in the C programming language.
- This directive is typically used to include header files, which typically contain global or commonly used definitions.

Example -`include Directive

```
// Include the file header.v, which contains declarations in the main verilog file
//design.v.
'include header.v
...
...
<Verilog code in file design.v>
...
...
```

EXERCISES

1. Practice writing the following numbers:

a) Decimal number 123 as a sized 8-bit number in binary. Use _ for readability.

b) A 16-bit hexadecimal unknown number with all x's.

c) A 4-bit negative 2 in decimal. Write the 2's complement form for this number.

d) An unsized hex number 1234.

ANS:

a) 123 = 8'b0111_1011

b) 16'hx

c) -4'd2=4'b1110

d) 32'h1234

EXERCISES

2. Are the following legal strings? If not, write the correct strings.

a) “This is a string displaying the % sign”

b) “out=in1+in2”

c) “Please ring a bell \007”

d) “This is a backslash \ character\n”

ANS:

a) “This is a string displaying the %% sign”

b) right

c) right

d) “This is a backslash \\ character”

EXERCISES

3. Are these legal identifiers ?

a) system1

b) 1reg

c) $latch

d) exec$

ANS:

a) right

EXERCISES

4. Declare the following variables in Verilog:

a) An 8-bit vector net called a_in.

b) A 32-bit storage register called address. Bit 31 must be the most significant bit. Set the value of the register to a 32-bit decimal number equal to 3.

c) An integer called cout.

d) A time variable called snap_shot.

e) An array called delays. Array contains 20 elements of the type integer.

f) A memory MEM containing 256 words of 64 bits each.

g) A parameter cache_size equal to 512.

ANS:

a) wire [7:0] a_in;

b) reg [31:0] address=32’d3;

c) integer cout;

d) time snap_shot;

e) integer delays [0:19];

f) reg [63:0] MEM [0:255];

g) parameter cache_size=512;

EXERCISES

5. What would be the output/effect of the following statements ?

a) latch = 4’d12;

$display(“The current value of latch = %b\n”, latch);

b) in_reg=3’d2;

$monitor($time, “ In register value = %b\n”, in_reg[2:0]);

c) `define MEM_SIZE 1024

$display(“ The maximum memory size is %h”, `MEM_SIZE);

ANS:

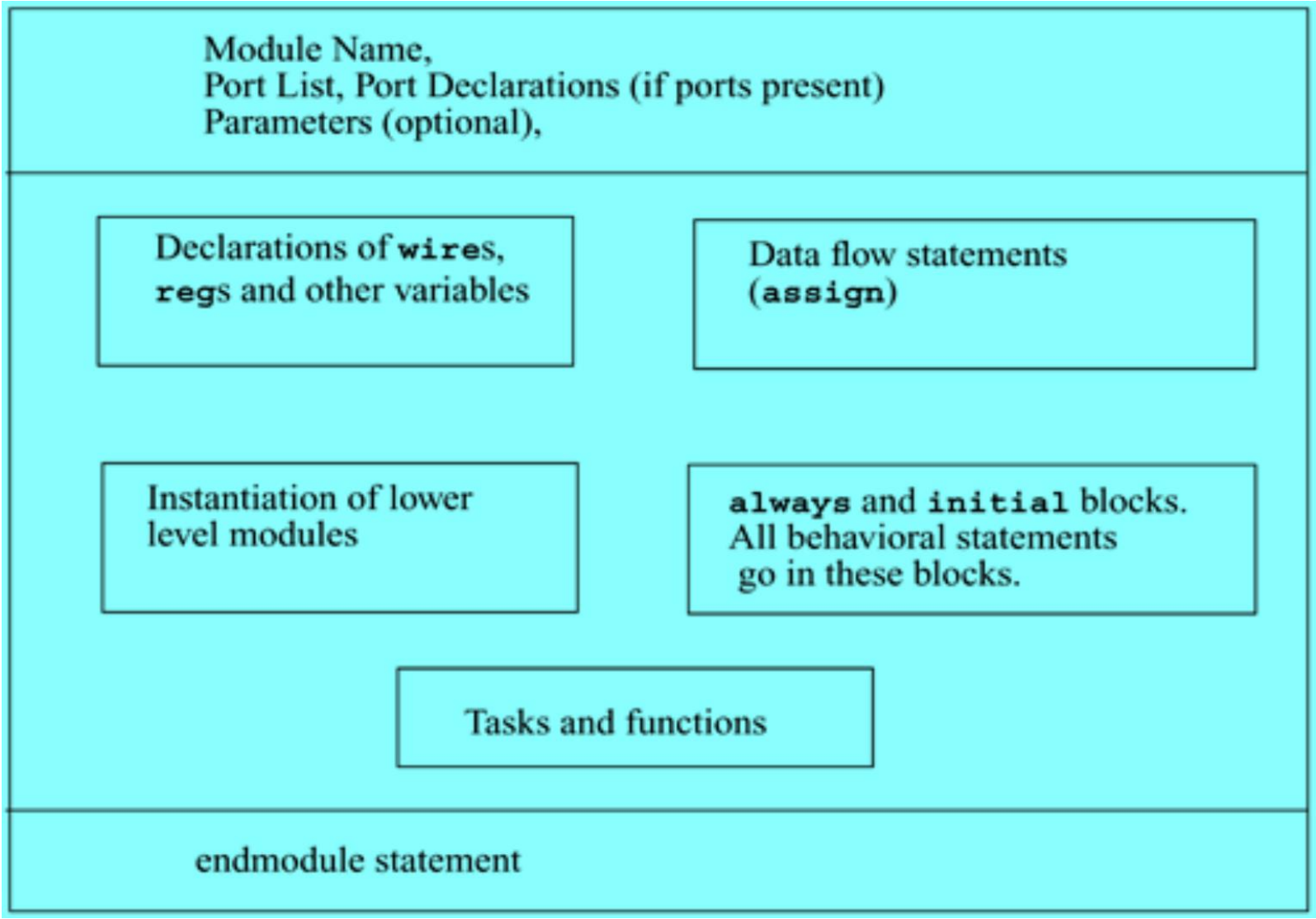
a) The current value of latch =4’b1100

b) 0 In register value = 3’b010

c) The maximum memory size is ‘h400

MODULES:

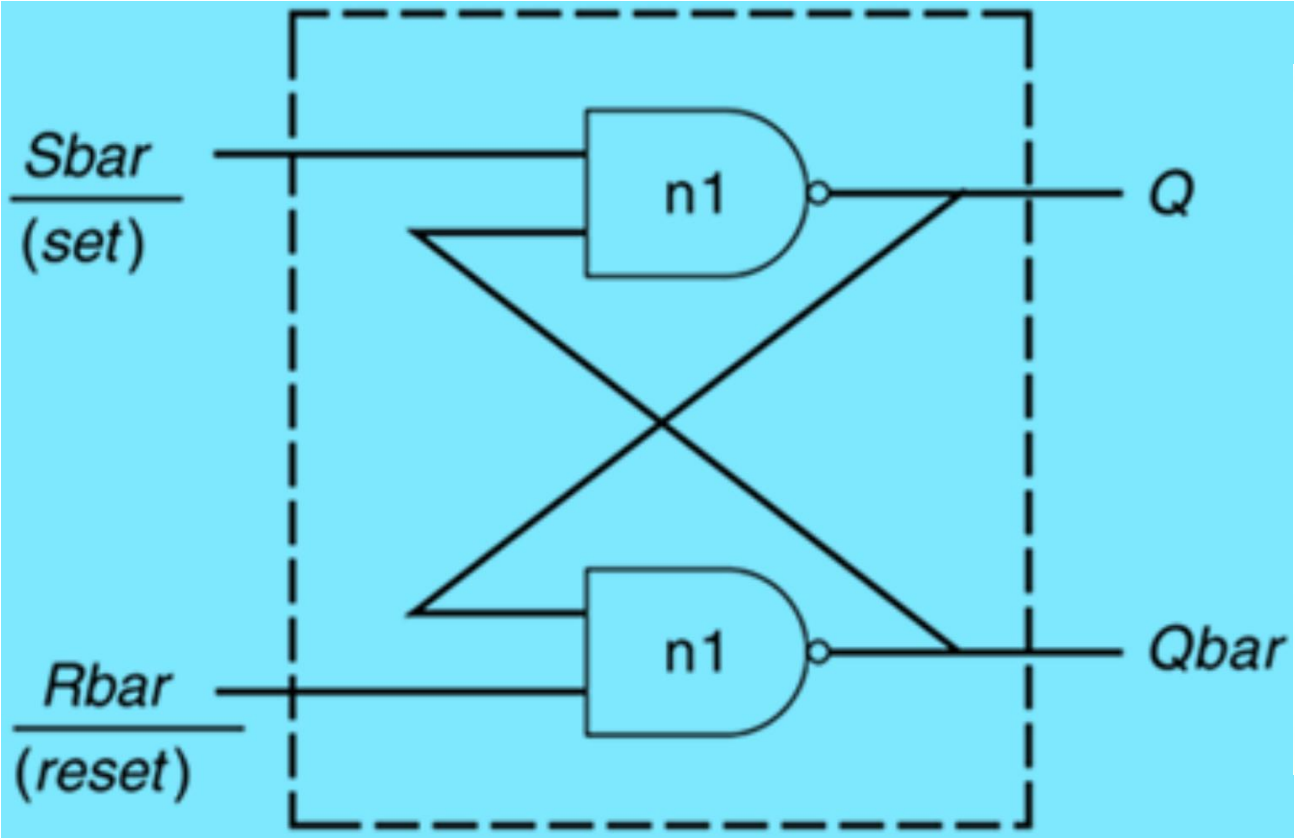
- **The internals of the module:** A module in Verilog consists of distinct parts,



MODULES:

- A module definition always begins with the keyword module.
- The module name, port list, port declarations, and optional parameters must come first in a module definition.
- Port list and port declarations are present only if the module has any ports to interact with the external environment.
- The five components within a module are: **variable declarations, dataflow statements, instantiation of lower modules, behavioural blocks, and tasks or functions.**
- These components can be in any order and at any place in the module definition.
- The endmodule statement must always come last in a module definition.
- All components except module, module name, and endmodule are optional and can be mixed and matched as per design needs.
- Verilog allows multiple modules to be defined in a single file.
- **The modules can be defined in any order in the file.**

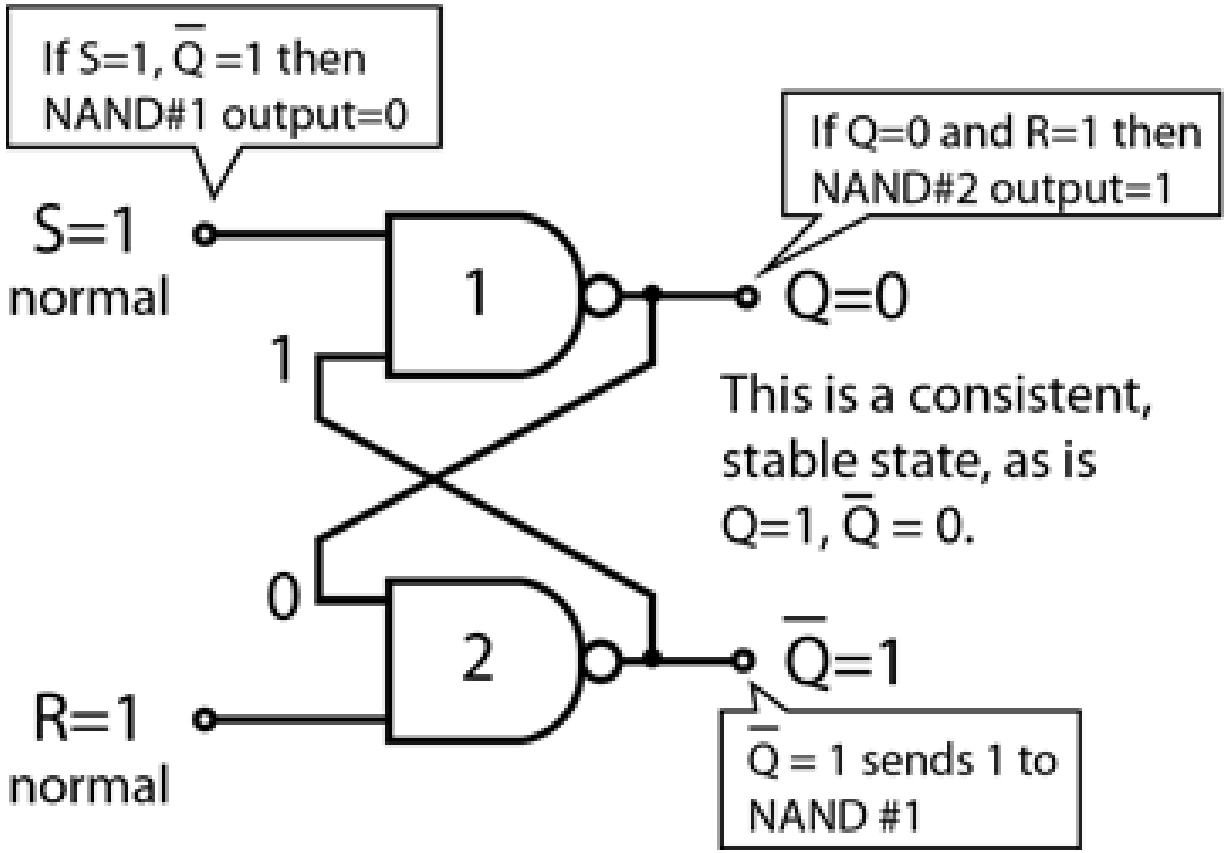
MODULES: EXAMPLE - AN SR LATCH



Sno	S	R	Q	Q'	State
1	1	0	1	0	Q is set to 1
2	1	1	1	0	No change
3	0	1	0	1	Q' is set to 1
4	1	1	0	1	No change
5	0	0	1	1	Invalid

The SR latch has S and R as the input ports and Q and Qbar as the output ports.

SR latch using NAND gates

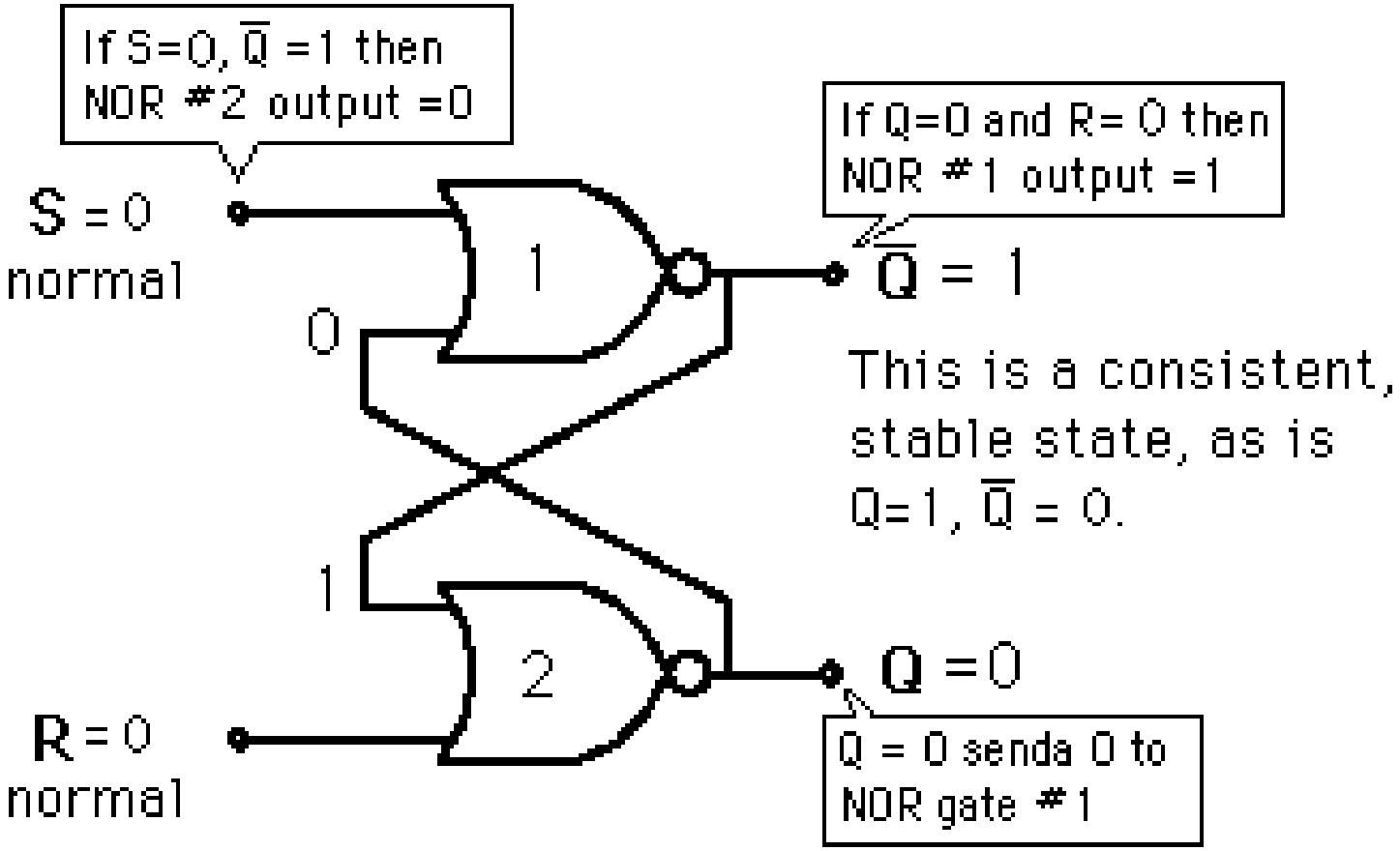


Truth Table

Set	Reset	Output
1	1	No change*
0	1	Q=1
1	0	Q=0
0	0	Invalid state

* can be used for data storage

SR latch using NOR gates



Truth Table

Set	Reset	Output
0	0	No change*
1	0	Q=1
0	1	Q=0
1	1	Invalid state

*can be used for data storage

MODULES: EXAMPLE - AN SR LATCH

```
// Module name and port list-  SR_latch module
module SR_latch(Q, Qbar, Sbar, Rbar); //Port declarations
output Q, Qbar;
input Sbar, Rbar;
// Instantiate lower-level modules,  In this case, instantiate
Verilog primitive nand gates. //Note, how the wires are
connected in a cross-coupled fashion.
nand n1(Q, Sbar, Qbar);
nand n2(Qbar, Rbar, Q);
Endmodule    // endmodule statement
```

MODULES: EXAMPLE - AN SR LATCH

```
// Stimulus module
module Top;
// Declarations of wire, reg, and other variables
wire q, qbar;
reg set, reset;
// Instantiate lower-level modules
// In this case, instantiate
//SR latch and Feed inverted set
//and reset signals to the SR latch
SR_latch m1(q, qbar, ~set, ~reset);
```

```
// Behavioral block, initial
initial
begin
$monitor($time, " set = %b, reset= %b, q= %b\n",set,reset,q);
set = 0; reset = 0;
#5 reset = 1;
#5 reset = 0;
#5 set = 1;
end
endmodule  // endmodule statement
```

MODULES: EXAMPLE - AN SR LATCH

Notice the following characteristics about the modules defined above:

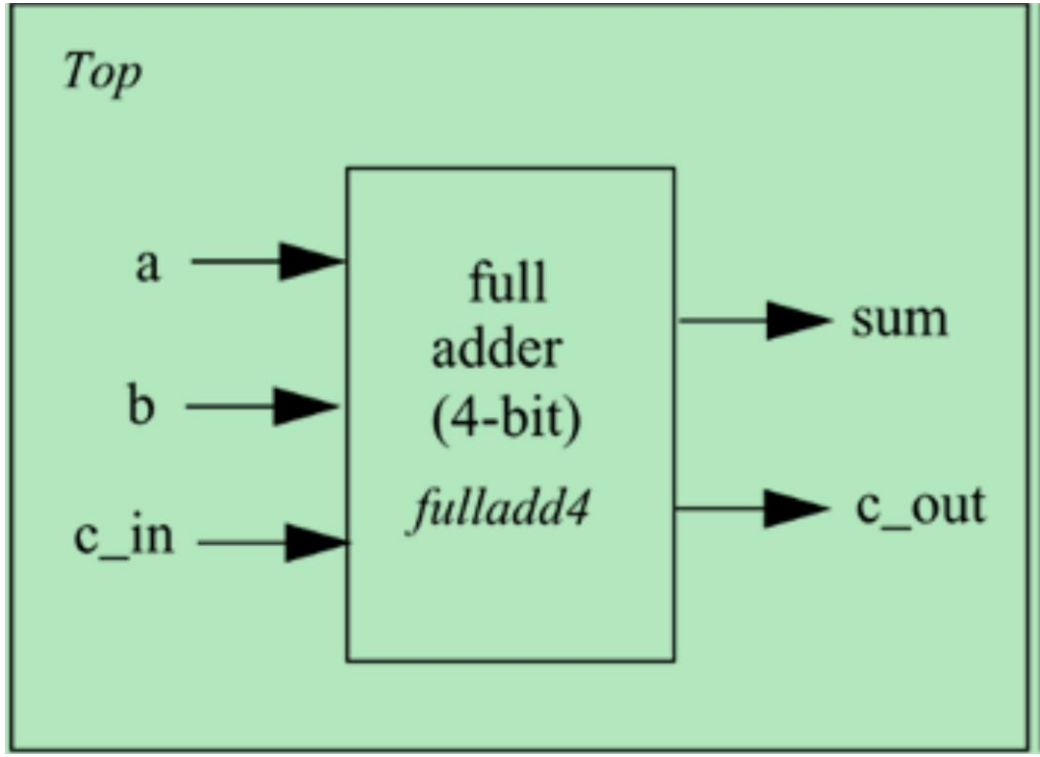
- In the SR latch definition above , notice that all components described in Figure need not be present in a module. We do not find variable declarations, dataflow (assign) statements, or behavioural blocks (always or initial).
- However, the stimulus block for the SR latch contains module name, wire, reg, and variable declarations, instantiation of lower level modules, behavioural block (initial), and endmodule statement but does not contain port list, port declarations, and data flow (assign) statements.
- Thus, all parts except module, module name, and endmodule are optional and can be mixed and matched as per design needs.

PORTS

- Ports provide the interface by which a module can communicate with its environment.
- For example, the input/output pins of an IC chip are its ports. The environment can interact with the module only through its ports.
- The internals of the module are not visible to the environment. This provides a very powerful flexibility to the designer.
- The internals of the module can be changed without affecting the environment as long as the interface is not modified.
- Ports are also referred to as terminals.

PORTS- **LIST OF PORTS**

- A module definition contains an optional list of ports.
- If the module does not exchange any signals with the environment, there are no ports in the list.



Example: List of Ports

```
module fulladd4(sum, c_out, a,b,c_in);
//Module with a list of ports
module Top;
// No list of ports, top-level module in simulation
```

PORTS- PORT DECLARATION

All ports in the list of ports must be declared in the module. Ports can be declared as follows:

Verilog Keyword	Type of Port
input	Input port
output	Output port
inout	Bidirectional port

PORTS- PORT DECLARATION

Example :Port Declarations

```
module fulladd4(sum, c_out, a, b, c_in);
//Begin port declarations section
output[3:0] sum;
output c_cout;
input [3:0] a, b;
input c_in;//End port declarations section
...
<module internals>
...
endmodule
```

PORTS- PORT DECLARATION

Note that all port declarations are implicitly declared as wire in Verilog.

Thus, if a port is intended to be a wire, it is sufficient to declare it as output, input, or inout.

Input or inout ports are normally declared as wires.

However, if output ports hold their value, they must be declared as reg.

Ports of the type input and inout cannot be declared as reg because reg variables store values and input ports should not store values but simply reflect the changes in the external signals they are connected to.

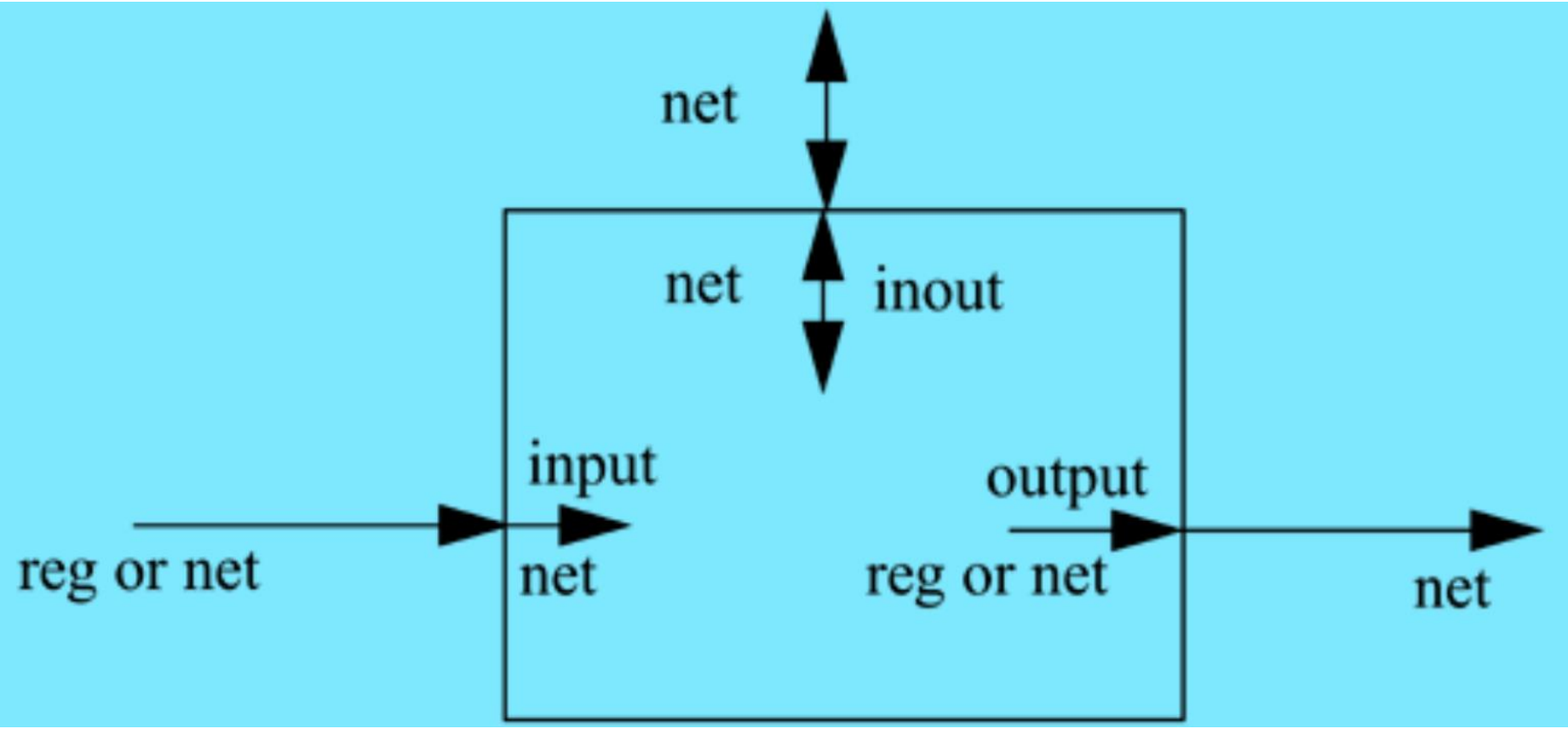
PORTS- PORT DECLARATION

Example : ANSI C Style Port Declaration Syntax

```
module fulladd4(output reg [3:0] sum,
output reg c_out,
input [3:0] a, b, //wire by default
input c_in); //wire by default
...
<module internals>
...
endmodule
```

PORT CONNECTION RULES

There are rules governing port connections when modules are instantiated within other modules.



- **Inputs**

Internally, input ports must always be of the type net. Externally, the inputs can be connected to a variable which is a reg or a net.

- **Outputs**

Internally, outputs ports can be of the type reg or net. Externally, outputs must always be connected to a net. They cannot be connected to a reg.

- **Inouts**

Internally, inout ports must always be of the type net. Externally, inout ports must always be connected to a net.

- **Width matching**

It is legal to connect internal and external items of different sizes when making intermodule port connections. However, a warning is typically issued that the widths do not match.

- **Unconnected ports**

Verilog allows ports to remain unconnected. For example, certain output ports might be simply for debugging, and you might not be interested in connecting them to the external signals.

CONNECTING PORTS TO EXTERNAL SIGNALS

Connecting by ordered list

- Connecting by ordered list is the most intuitive method for most beginners.
- The signals to be connected must appear in the module instantiation in the same order as the ports in the port list in the module definition.

Port Declarations

```
module fulladd4(sum, c_out, a, b, c_in);
//Begin port declarations section
output[3:0] sum;
output c_cout;
input [3:0] a, b;
input c_in;
//End port declarations section
...
<module internals>
...
endmodule
```

Connection by Ordered List

```
module Top;
//Declare connection variables
reg [3:0]A,B;
reg C_IN;
wire [3:0] SUM;
wire C_OUT;
//Instantiate fulladd4, call it fa_ordered.
//Signals are connected to ports in order (by position)
fulladd4 fa_ordered(SUM, C_OUT, A, B, C_IN);
...
<stimulus>
...
endmodule
```

CONNECTING PORTS TO EXTERNAL SIGNALS

Connecting ports by name

For large designs where modules have, say, 50 ports, remembering the order of the ports in the module definition is impractical and error-prone.

Verilog provides the capability to connect external signals to ports by the port names, rather than by position.

Another advantage of connecting ports by name is that as long as the port name is not changed, the order of ports in the port list of a module can be rearranged without changing the port connections in module instantiations.

```
// Instantiate module fa_byname and connect signals to ports by name

fulladd4 fa_byname(.c_out(C_OUT), .sum(SUM), .b(B),.c_in(C_IN),.a(A),);
```

EXERCISES

1. What are the basic components of a module? Which components are mandatory?

ANS:

a) Module Name, Port List, Port Declarations, Parameters, Declarations of wires, regs and other variables, Data flow statements, Instantiation of lower level modules, always and initial blocks, Tasks and functions, endmodule statement.

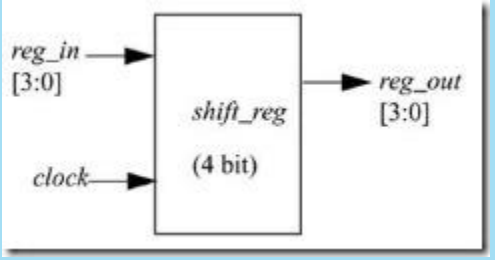
b) module, module name, and endmodule.

2. Does a module that does not interact with its environment have any I/O ports? Does it have a port list in the module definition?

ANS:

a) have b) no

3. A 4-bit parallel shift register has I/O pins as shown in the figure bellow. Write the module definition for this module shift_reg. Include the list of ports and port declarations. You do not need to show the internals.



ANS:

```
module shift_reg(reg_out,reg_in,clock);

output reg [3:0]reg_out;

input [3:0]reg_in;

input clock;

….

endmodule
```

EXERCISES

4. Declare a top-level module stimulus. Define REG_IN(4-bit)and CLK(1-bit)as reg register variables and REG_OUT(4-bit)as wire. Instanitiate the module shift_reg and call it sr1. Connect the ports by ordered list.

ANS:

EXERCISES

5. Connect the ports in Step 4 by name.

ANS: shift_reg(.reg_out(REG_OUT),.reg_in(REG_IN),.clock(CLK));

6.Write the hierarchical names for variables REG_IN,CLK,REG_OUT;

ANS: stimulus.REG_IN, stimulus.CLK, stimulus.REG_OUT

7. Write the hierarchical name for the instance sr1. Write the hierarchical names for its ports clock and reg_in.

ANS:

stimulus.sr1,

stimulus.sr1.clock

stimulus.sr1.reg_in

SOME MORE..

Differentiate between the following: (a) Register and wire data types

1. Wire (Combinational Logic)

•**Usage**: Used for combinational logic, where the output is dependent on the current input.

•**Behavior**: Wires do not store values; they are used to connect different parts of the circuit, passing signals between them.

•**Declaration**: wire keyword.

•**Assignment**: Continuous assignment with assign keyword.

Eg: wire out;

assign out = in1 & in2; // Continuous assignment

2. Register (Sequential Logic)

•**Usage**: Used in sequential logic and stores values (acts as a storage element like a flip-flop or latch).

•**Behavior**: Can hold a value until it is explicitly changed in procedural blocks like always.

•**Declaration**: reg keyword.

•**Assignment**: Assigned values inside a procedural block such as always or initial.

•**Eg**:**reg out;**

```
always @(posedge clk)
Begin
 out <= in1 & in2; // Sequential assignment (clocked)
```

Differentiate between the following: (b) $monitor and $display system tasks

1. $display System Task

•**Usage**: Used to print information once when it is executed.

•**Behavior**: It outputs the values of variables or signals at the moment it is called, then stops. If you want continuous updates, you would have to place $display inside an always block that triggers repeatedly.

•**Output**: Prints a line of text or the current values of variables/signals **one time**.

•**Use case**: Best for printing status at specific points, such as debugging the value at certain time steps.

Example:

```
initial begin
$display("Time: %0t, A = %b, B = %b", $time, A, B);
end
```

2. $monitor System Task

•**Usage**: Used to continuously monitor signals and print their values whenever any of the variables in the list changes.

•**Behavior**: After being called, $monitor automatically outputs updated values whenever any signal in its argument list changes. It is designed for continuous monitoring throughout the simulation.

•**Output**: Continuously prints values whenever there is a change in the monitored signals.

• **Use case:** Best for tracking the values of variables or signals throughout the entire simulation without repeatedly writing multiple $display statements.

•**Example**:

```
initial begin
  $monitor("Time: %0t, A = %b, B = %b", $time, A, B);
End
```

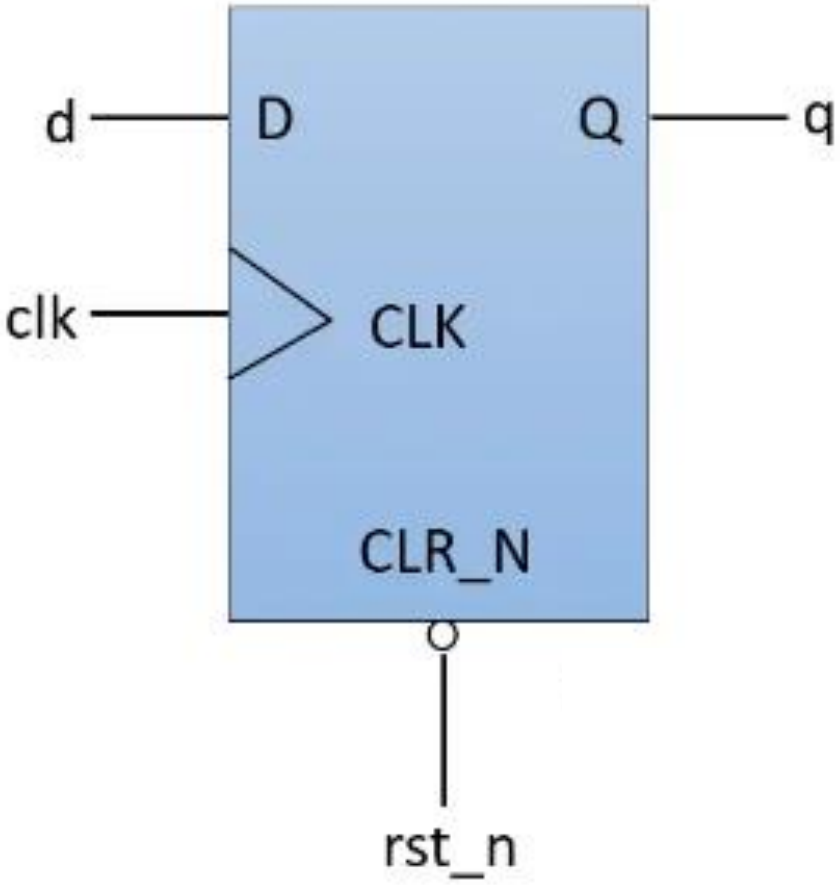
Differentiate between the following: (c) $finish and $stop

Aspect	$finish	$stop
Effect	Completely terminates the simulation	Pauses the simulation temporarily
Simulation Behavior	Cannot resume after calling $finish	Can resume after calling $stop
Purpose	Used for ending the simulation	Used for debugging and inspection
Use case	End of simulation after a test	Pausing simulation for checks/debugging

Example:	initial begin
	#100;
	$finish; // The simulation will end at time 100
	end
		initial begin
		#50;
		$stop; // The simulation will stop (pause) at time 50
		end

Qn. Model the verilog code for D flipflop with asynchronous active low input (reset).

The D flip flop is a basic sequential element that has data input ‘d’ being driven to output ‘q’ as per clock edge. Also, the D flip-flop held the output value till the next clock cycle. Hence, it is called an edge-triggered memory element that stores a single bit.



The D flip flop shown here is positive edge-triggered and has asynchronous active low reset.

As soon as reset is triggered, the output gets reset

VERILOG CODE

```
module D_flipflop (  input clk, rst_n,  input d,  output reg q  );
   always@(posedge clk or negedge rst_n)
    begin
    if(!rst_n)
    q <= 0;
    else
    q <= d;
  end
endmodule
```

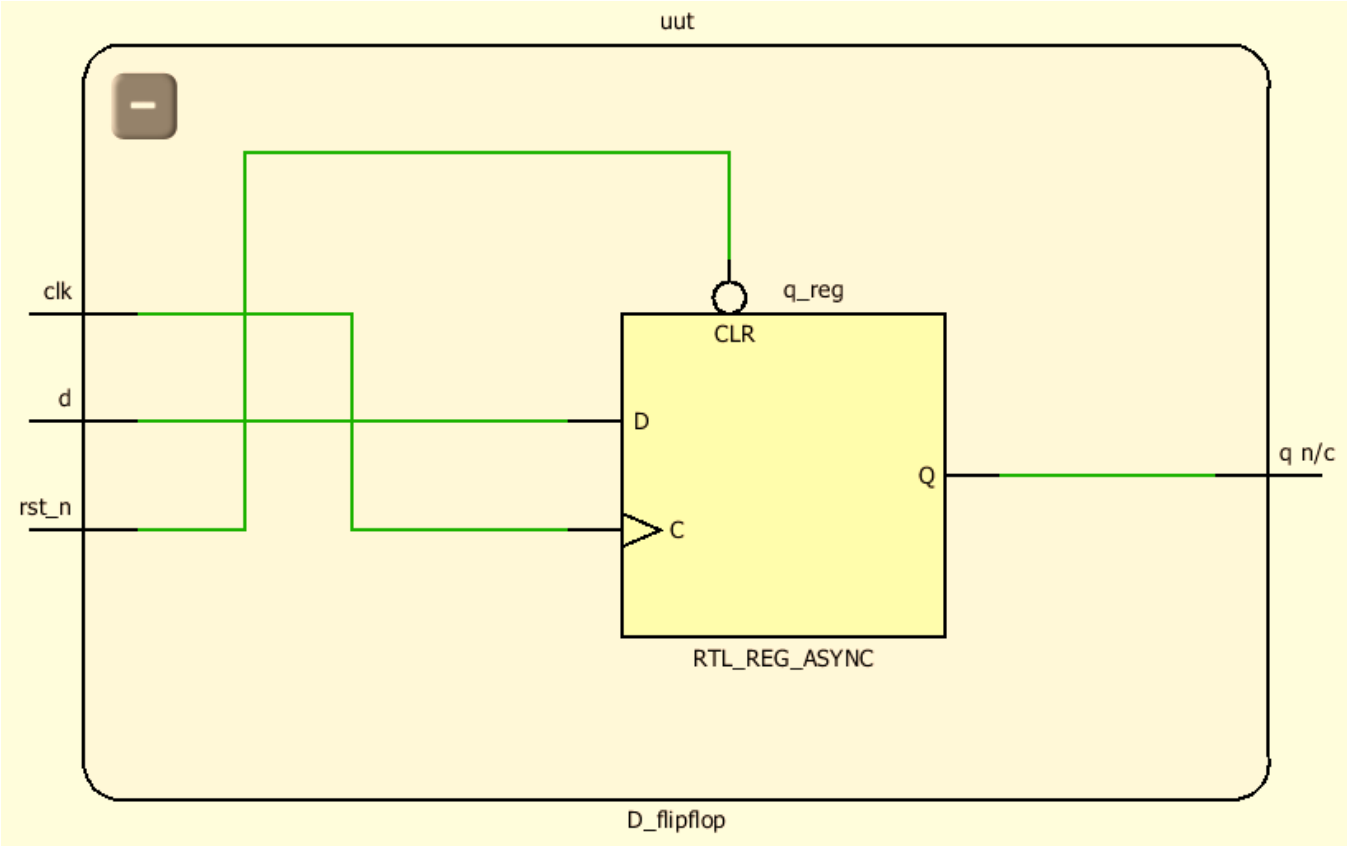
TESTBENCH CODE

```
module tb_D_flipflop;    // Declare testbench signals

  reg clk;        // Clock signal

  reg rst_n;      // Asynchronous active-low reset signal

  reg d;          // D input

  wire q;         // Q output

  // Instantiate the D flip-flop module

D_flipflop uut (  .clk(clk),  .rst_n(rst_n),  .d(d),  .q(q) );

initial begin // Clock generation

    clk = 0;

    forever #5 clk = ~clk;

  end
```

```
 // Stimulus block
  initial begin
    // Initial values
    d = 0;
    rst_n = 0;  // Reset is active (low)

    // Release reset after 10 time units
    #10;     rst_n = 1;  // Release reset
// Test cases:
    #10;     d = 1;  // Apply 1 to D

    #10;     d = 0;  // Apply 0 to D

    #10;     d = 1;  // Apply 1 to D again

    #10;     rst_n = 0;  // Assert reset again
    #10;     rst_n = 1;  // Release reset
 #20;
   $finish;
  end
endmodule
```

Simulation Output

Schematic Diagram



Qn. Deduce the design block for the given Verilog code and draw the waveforms for the stimulus given:

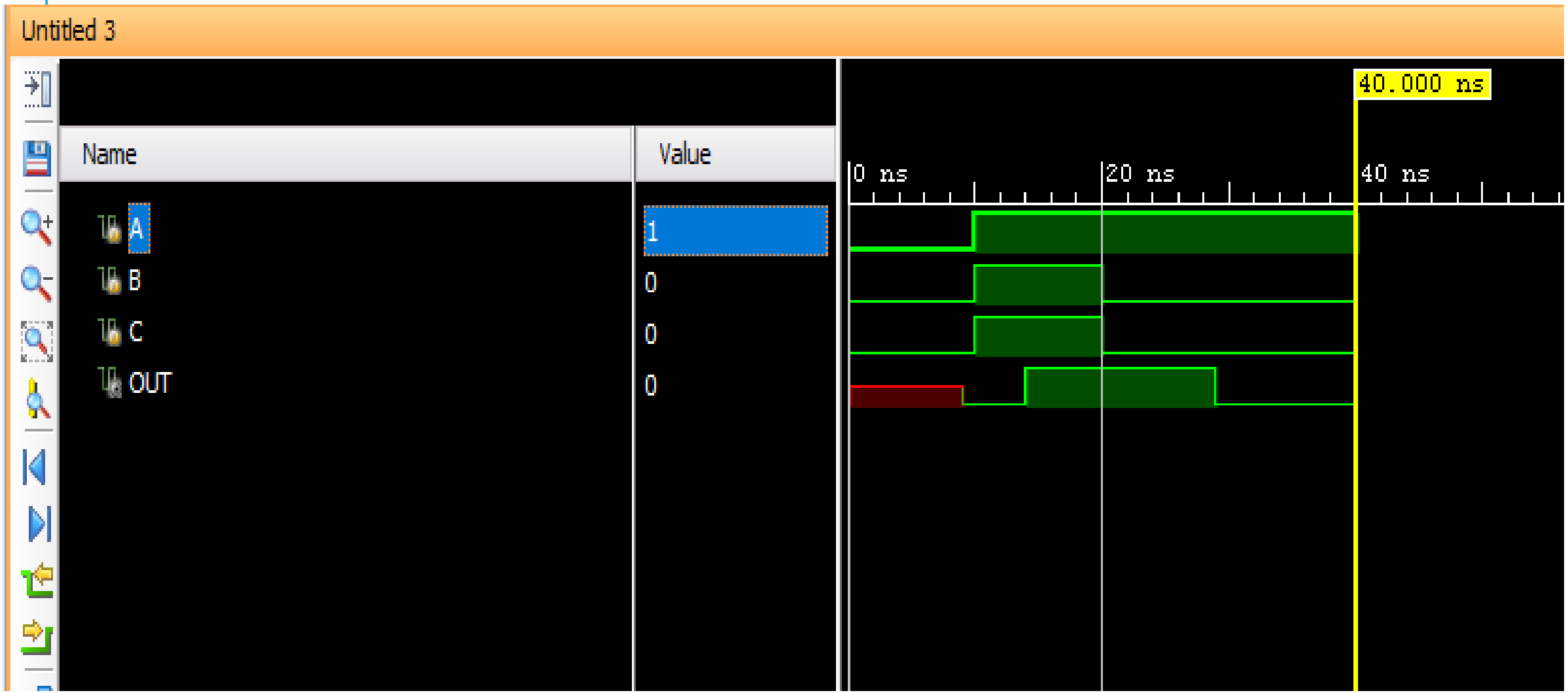
```
module example(out, a, b, c);

input a, b, c;

output out;

and #5 a1(e, a, b);

or #4 a2(out, e, c);

endmodule
```

```
module stimulus;
reg A, B, C;
wire OUT;
example ex1(OUT, A, B, C);
initial
begin
A = 1’b0, B = 1’b0, C = 1’b0;
#10 A = 1’b1, B = 1’b1, C = 1’b1;
#10 A = 1’b1, B = 1’b0, C = 1’b0;
#20 $finish;
end
endmodule
```

ANS:

```
module stimulus;
    reg A, B, C;
    wire OUT;
    example ex1( A, B, C,OUT);
    initial
    begin
    A = 1'b0; B = 1'b0; C = 1'b0;
    #10 A = 1'b1; B = 1'b1; C = 1'b1;
    #10 A = 1'b1; B = 1'b0; C = 1'b0;
    #20 $finish;
    end
endmodule
```

Simulation Output



Thank you!